

Ανίχνευση Δικτυακών Εισβολών με Μηχανική Μάθηση

Ανάλυση του Συνόλου Δεδομένων CIC-IDS-2017

Εξόρυξη Δεδομένων & Μηχανική Μάθηση

Βασίλειος Μπίτζας 1083796 up1083796@ac.upatras.gr

Τμήμα Μηχανικών Ηλεκτρονικών Υπολογιστών & Πληροφορικής
Πανεπιστήμιο Πατρών

18 Ιουνίου 2026

Περιεχόμενα

1	Εισαγωγή	8
1.1	Πλαίσιο και Κίνητρο	8
1.2	Στόχοι Εργασίας	8
1.3	Περιγραφή Δεδομένων	8
1.3.1	CIC-IDS-2017 Dataset	8
1.3.2	Συνοπτικά Μεγέθη	9
1.4	Διάρθρωση της Αναφοράς	9
2	Θεωρητικό Υπόβαθρο	11
2.1	Προεπεξεργασία Δεδομένων	11
2.1.1	Missing Values και Infinities	11
2.1.2	Ακριβή Διπλότυπα	11
2.2	Κλάσεις σε Ανισορροπία	11
2.3	Pearson Correlation	12
2.3.1	Τυφλά Σημεία	12
2.4	Multicollinearity	12
2.5	log1p Transform και Skewness	13
2.6	Stratified Sampling για Οπτικοποίηση	13
2.7	Αξιολόγηση Ταξινομητών (Q2)	13
2.7.1	Confusion Matrix	13
2.7.2	Precision, Recall, F1	13
2.7.3	Grid Search Cross-Validation	14
2.8	Συσταδοποίηση (Q3)	14
2.8.1	K-Means	14
2.8.2	DBSCAN	14
2.8.3	Hierarchical Clustering	14
2.8.4	External Validation με ARI	14
3	Ερώτημα 1: Exploratory Data Analysis	15
3.1	Επισκόπηση του Pipeline	15
3.2	Φόρτωση και Επισκόπηση	15
3.2.1	Ενοποίηση των 8 CSV	15
3.2.2	Καθαρισμός Ετικετών	15
3.3	Καθαρισμός Δεδομένων	16
3.3.1	Infinities και Missing Values	16
3.3.2	Duplicates	16
3.4	Κατανομή Κλάσεων	16
3.5	Περιγραφικά Στατιστικά	18
3.6	Οπτικοποιήσεις	18
3.6.1	Ιστογράμματα Top Features	18
3.6.2	Boxplots	18
3.6.3	Correlation Heatmap	19
3.6.4	Top Correlated Pairs	20

3.6.5	Correlation με το Label	21
3.7	Feature Selection	21
3.7.1	Κριτήρια Αφαίρεσης	21
3.7.2	Αποτελέσματα	22
3.7.3	Τελική Λίστα Features	22
3.8	Αποθήκευση του Clean Snapshot	22
3.9	Σύνοψη Q1	23
4	Ερώτημα 2: Ταξινόμηση	24
4.1	Μεθοδολογία	24
4.1.1	Balanced Subsampling σε Target Μέγεθος	24
4.1.2	Encoding, Scaling και Imbalance Handling	24
4.1.3	Πρωτόκολλο Splits	25
4.1.4	Scoring Metric για την Επιλογή Hyperparameters	25
4.2	Logistic Regression	25
4.3	Decision Tree	27
4.4	Random Forest	28
4.5	Συγκριτική Ανάλυση	30
4.5.1	Σύγκριση Balanced vs Proportional Sampling	31
4.5.2	Απαντήσεις στις Τέσσερις Ερωτήσεις της Εκφώνησης	32
4.6	Σύνοψη Q2	34
5	Ερώτημα 3: Συσταδοποίηση	35
5.1	Preprocessing και PCA	35
5.2	K-Means	36
5.3	Hierarchical (Agglomerative)	38
5.4	DBSCAN	40
5.5	Συγκριτική Ανάλυση	42
5.5.1	Απαντήσεις στις Ερωτήσεις της Εκφώνησης	42
5.5.2	Σύγκριση Proportional vs Balanced Sampling (mirror του Q2)	43
5.6	Σύνοψη Q3	45
6	Συμπεράσματα	46
6.1	Σύνοψη Όλου του Αγωγού	46
6.2	Συμπεράσματα ανά Ερώτημα	46
6.2.1	Q1 - Δομή και Καθαρισμός των Δεδομένων	46
6.2.2	Q2 - Supervised Classification	46
6.2.3	Q3 - Unsupervised Clustering	47
6.3	Συνολικά Διδάγματα και Best Practices	47
6.4	Πιθανές Επεκτάσεις	47
Α'	Οδηγίες Εγκατάστασης και Εκτέλεσης	49
Α'.1	Απαιτήσεις	49
Α'.2	Εγκατάσταση	49
Α'.3	Εκτέλεση του Q1	49
Α'.4	Εκτέλεση των Q2 και Q3	50
Α'.5	Αναπαραγωγικότητα	50
Β'	Επιτάχυνση Υλικού και Βελτιστοποίηση	51

B'.1	Hardware και Επισκόπηση	51
B'.2	Το Βασικό Δίδαγμα: οι Accelerators δεν είναι πάντα Ισοδύναμοι	51
B'.3	Μετρήσεις Επιτάχυνσης	52
B'.4	Όρια Κλιμάκωσης (Scaling Walls)	52
B'.5	Setup του RAPIDS cuML (WSL2) – Αναπαραγωγή	52
B'.6	Στρατηγική Δειγματοληψίας	53

Κατάλογος Σχημάτων

3.1	Κατανομή κλάσεων στο καθαρισμένο CIC-IDS-2017. Το αριστερό σχήμα χρησιμοποιεί λογαριθμικό άξονα ώστε να φαίνονται και οι σπάνιες κλάσεις (Heartbleed, Sql Injection, Infiltration).	17
3.2	Ιστογράμματα με KDE για τις 20 στήλες μεγαλύτερης διασποράς (log1p scale). Οι περισσότερες κατανομές είναι έντονα λοξές και heavy-tailed.	18
3.3	Boxplots (log1p scale) για τις 20 στήλες μεγαλύτερης διασποράς. Τα outliers πάνω από τα whiskers είναι διαγνωστικά για επιθέσεις DoS/DDoS και δεν αφαιρούνται.	19
3.4	Pearson correlation heatmap των 77 αριθμητικών features. Τα μεγάλα κόκκινα clusters αντιστοιχούν σε αλγοριθμικές ταυτότητες του CICFlowMeter (π.χ. Subflow Fwd Bytes $\approx$ Total Length of Fwd Packets).	20
4.1	Logistic Regression (Σενάριο B) - heatmap των συντελεστών για τα top-15 features (κατά μέσο $ \text{coef} $ σε όλες τις κλάσεις). Έντονο κόκκινο = το feature οδηγεί strongly προς αυτή την κλάση· έντονο μπλε = αντενδείκνυται. Παράδειγμα: η PortScan έχει σαφώς θετικούς συντελεστές στα Bwd Packet Length Min και Bwd Packets/s (μικρά backward packets σε υψηλό rate - υπογραφή scanning).	26
4.2	Decision Tree (Σενάριο B, actual best) - top 3 levels. Το πραγματικό δέντρο μπορεί να φτάνει σε πολύ μεγαλύτερο βάθος, αλλά τα πρώτα splits χωρίζουν τις περισσότερες κλάσεις.	28
4.3	Standalone shallow Decision Tree (max_depth = 3, entropy) εκπαιδευμένο ξεχωριστά για clean visualization. Όχι το best model - απλώς δείχνει πώς θα μοιάζει ένα ολόκληρο δέντρο 3 επιπέδων.	28
4.4	Top-20 feature importances του Random Forest (Σενάριο B).	29
4.5	Random Forest (Σενάριο B) - top 3 levels του πρώτου voter του δάσους (από τα 100 δέντρα συνολικά). Τα splits εδώ διαφέρουν από αυτά του standalone DT - σημείο της feature randomness που επιβάλλει το RF.	30
4.6	F1-weighted έναντι F1-macro και για τις 6 παραλλαγές. Η διαφορά $F1_w - F1_m$ είναι η «υπογραφή του imbalance»: πόσο «χρωστά» το μοντέλο την απόδοσή του στις μεγάλες κλάσεις.	31
4.7	Random Forest (Σενάριο B) - row-normalized confusion matrix. Οι διαγώνιοι κοντά στο 1 δείχνουν per-class recall· οι off-diagonal entries δείχνουν confusion patterns.	33
4.8	Logistic Regression (Σενάριο B) - row-normalized confusion matrix. Σημαντικά χειρότερη ποιότητα στις Web-Attack και Bot κλάσεις σε σχέση με το RF.	34
5.1	PCA explained variance: per-component (μπλε bars) και cumulative (κόκκινη γραμμή). Με 19 components καλύπτουμε $\sim 86,3\%$ της variance· επιλέγουμε αυτό ως input για όλους τους αλγορίθμους clustering. Παρατηρήστε ότι ο πρώτος component συλλαμβάνει μόνο $\sim 18\%$: δεν υπάρχει ένα κυρίαρχο axis. Η information είναι κατανομημένη σε πολλά components.	36
5.2	K-Means - inertia (μπλε) και silhouette (κόκκινο) ανά k . Το silhouette κορυφώνεται στο $k = 2$ (normal vs volumetric attacks)· το $k = 15$ (= #labels) τρέχει ως reference.	37

5.3	K-Means $k = 2$ (best) - PCA 2-D scatter (rendering sample 50K points). Αριστερά: colored by cluster. Δεξιά: colored by true Label. Με 83% BENIGN, ο χώρος κυριαρχείται από το «κανονικό» cloud· η διχοτόμηση normal-vs-volumetric είναι ο πιο ευδιάκριτος διαχωρισμός.	38
5.4	K-Means $k = 2$ - cluster composition heatmap. Row-normalized. c0 : $\sim 89\%$ BENIGN (normal traffic). c1 : $\sim 49\%$ DoS Hulk + $\sim 12\%$ DDoS (volumetric attacks). Ένας ερμηνεύσιμος, αν και χονδρικός, διαχωρισμός.	38
5.5	Dendrogram (ward linkage, top 30 merges). Φαίνεται καθαρά διπλή ιεραρχική δομή: ένα μεγάλο σύνολο merges σε χαμηλό height (πορτοκαλί - κανονικό traffic και κοντινά attacks) και ένα απομονωμένο cluster (πράσινο - probably attack patterns που διαφέρουν δραματικά). Η μεγάλη κάθετη γραμμή στο height ~ 800 δείχνει την τελική συγχώνευση των δύο sub-trees.	39
5.6	Hierarchical (best linkage, $k = 2$) - PCA 2-D scatter στο 2.468-row subsample.	40
5.7	Hierarchical (best linkage, $k = 2$) - cluster composition heatmap.	40
5.8	k-distance graph ($k = 4$) πάνω σε στρωμένο sample 20.000 rows. Η κύρια καμπύλη παραμένει σχεδόν επίπεδη μέχρι το $\sim 80\%$ percentile και έπειτα εκτοξεύεται απότομα: σαφής αγκώνας στο $\varepsilon \approx 0,3-0,5$, κάτι που επιβεβαιώνει το sweep μας.	41
5.9	DBSCAN ($\varepsilon = 1,5$, min_samples = 5) - PCA 2-D scatter. Τα grey x-markers είναι <i>noise points</i> (2.649 rows, $\sim 0,5\%$ σε αυτό το config· στα μικρά ε το noise εκτοξεύεται σε 30K-51K).	42

Κατάλογος Πινάκων

1.1	Βασικά μεγέθη του CIC-IDS-2017 πριν και μετά την προεπεξεργασία.	9
3.1	Προοδευτική μείωση του μεγέθους του dataset κατά τον καθαρισμό.	16
3.2	Κατανομή των 15 κλάσεων μετά τον καθαρισμό. Το imbalance ratio (max/min) είναι $\approx 195\,173\times$	17
3.3	Top-15 ζεύγη με $ r > 0.997$. Τα 8 πρώτα είναι αλγοριθμικές ταυτότητες του CICFlowMeter.	21
3.4	Τα τρία κριτήρια feature selection και η αιτιολογία τους.	21
3.5	Οι 31 στήλες που αφαιρέθηκαν στο feature selection.	22
3.6	Βασικά αποτελέσματα του Q1.	23
4.1	Logistic Regression - test metrics (500K balanced).	26
4.2	Decision Tree - test metrics (500K balanced).	27
4.3	Random Forest - test metrics (500K balanced).	29
4.4	Q2 - σύγκριση 6 παραλλαγών στο ίδιο test set (100.000 rows, 500K balanced run).	30
4.5	Επίδραση της στρατηγικής δειγματοληψίας στο F1-macro (και τα δύο runs στο ίδιο TARGET_ROWS=500_000). Το LR κερδίζει +0,19 και το RF +0,07 με balanced· το DT μένει πρακτικά ίδιο (μάλιστα οριακά υψηλότερο στο proportional, 0,8946 vs 0,8705 - ένα βαθύ δέντρο πιάνει τις rare κλάσεις ακόμα κι όταν είναι λίγες). Επιβεβαιώνει τον κανόνα: για supervised classification το training set πρέπει να αναπαριστά τα <i>σύνορα</i> των κλάσεων, όχι το πραγματικό prior.	31
4.6	Q2 - σύγκριση 6 παραλλαγών στο proportional test set (100.000 rows, 500K proportional run· τα δύο σενάρια A/B έδωσαν ταυτόσημα best params και test metrics ανά μοντέλο).	32
4.7	Βασικά αποτελέσματα του Q2 (500K balanced run).	34
5.1	K-Means - test metrics (proportional 500K). Best $k = 2$ έχει το υψηλότερο silhouette· το $k = 15$ reference έχει καλύτερο (χαμηλότερο) Davies-Bouldin - οι δύο μετρικές διαφωνούν σε ασύμμετρα δεδομένα.	37
5.2	Hierarchical Clustering - σύγκριση linkage methods ($k = 2$, 2.468-row stratified subsample). Best: complete/average (ταυτόσημα).	39
5.3	DBSCAN sweep (proportional 500K) - όλες οι 15 παραλλαγές. Best silhouette: $\varepsilon = 1,5$, <code>min_samples = 5</code> (μόλις 0,105). Best Davies-Bouldin: $\varepsilon = 1,5$, <code>min_samples = 20</code> . Όλα τα silhouette είναι χαμηλά (αρκετά αρνητικά στα μεγάλα ε): το DBSCAN δυσκολεύεται σε αυτό το πυκνό, ανομοιόμορφο dataset.	41
5.4	Q3 - σύγκριση best configurations ανά αλγόριθμο (proportional 500K). <i>Caveats</i> : το Hierarchical εκπαιδεύτηκε σε 2.468-row subsample λόγω $O(n^2)$ memory, και το υψηλό του silhouette σε $k = 2$ αντανακλά εύκολο διαχωρισμό, όχι αντιστοίχιση με labels.	42
5.5	Q3 balanced run (BALANCED_SAMPLING=True, 500K) - σύγκριση best configurations ανά αλγόριθμο. Best- k μετατοπίζεται από 2 (proportional) σε 12. Το Hierarchical εκπαιδεύτηκε σε 2.468-row subsample.	44
5.6	Βασικά αποτελέσματα του Q3 (proportional 500K run).	45
B.1	Ενδεικτικές μετρήσεις. Το DBSCAN στη GPU είναι $\sim 100\times$ ταχύτερο στο χειρότερο config· τα PCA/K-Means σχεδόν στιγμιαία ακόμη και σε 1M rows (peak VRAM $\sim 1,4$ GB).	52

Κεφάλαιο 1

Εισαγωγή

1.1 Πλαίσιο και Κίνητρο

Η ραγδαία αύξηση της εξάρτησης των υποδομών από το διαδίκτυο έχει καταστήσει την **ανίχνευση δικτυακών εισβολών** (Network Intrusion Detection) έναν από τους πιο κρίσιμους τομείς της κυβερνοασφάλειας. Τα σύγχρονα συστήματα ανίχνευσης εισβολών (Intrusion Detection Systems, IDS) δεν μπορούν πλέον να βασίζονται αποκλειστικά σε *signature-based* κανόνες, καθώς οι επιτιθέμενοι προσαρμόζουν συνεχώς τις τεχνικές τους. Αντίθετα, οι προσεγγίσεις που βασίζονται σε **μηχανική μάθηση** (machine learning) μαθαίνουν μοτίβα από ιστορικά δεδομένα δικτυακής ροής και γενικεύουν σε νέες, άγνωστες επιθέσεις.

Η παρούσα εργασία αναπτύσσει ένα ολοκληρωμένο pipeline εξόρυξης δεδομένων πάνω στο **CIC-IDS-2017**, ένα από τα πιο ευρέως χρησιμοποιούμενα δημόσια σύνολα δεδομένων για έρευνα IDS. Το σύνολο περιέχει **2.83 εκατομμύρια** εγγραφές δικτυακής ροής (flow records) κατανεμημένες σε μία κανονική κλάση (**BENIGN**) και 14 διαφορετικούς τύπους επιθέσεων, από DDoS και port-scanning μέχρι web attacks και εξαιρετικά σπάνια exploits όπως το Heartbleed.

1.2 Στόχοι Εργασίας

Η εργασία δομείται γύρω από τρία συνδεδεμένα ερωτήματα:

1. **Ερώτημα 1 - Exploratory Data Analysis.** Φόρτωση, καθαρισμός και στατιστική περιγραφή του CIC-IDS-2017. Αντιμέτωπιση Inf/NaN, διπλοτύπων, ανισορροπίας κλάσεων και πολυσυγγραμμικότητας features. Εξαγωγή ενός καθαρού, cached snapshot για επαναχρησιμοποίηση στα Q2 και Q3.
2. **Ερώτημα 2 - Ταξινόμηση.** Εκπαίδευση και σύγκριση τριών αλγορίθμων (Logistic Regression, Decision Tree, Random Forest) με δύο διαφορετικές ρυθμίσεις Grid Search (A και B) για hyperparameter tuning. Αξιολόγηση με macro-F1 και per-class μετρικές λόγω έντονης ανισορροπίας.
3. **Ερώτημα 3 - Συσταδοποίηση.** Εφαρμογή K-Means, Hierarchical Clustering και DBSCAN σε unsupervised mode. Σύγκριση των συστάδων με τις γνωστές ετικέτες (external validation) για να εκτιμηθεί κατά πόσο η δομή των δεδομένων υποστηρίζει ανίχνευση επιθέσεων χωρίς labels.

Η αναφορά αυτή επικεντρώνεται κυρίως στο **Ερώτημα 1**, το οποίο αποτελεί το υπόβαθρο για τα υπόλοιπα δύο. Τα Q2 και Q3 θα προστεθούν ως ξεχωριστά κεφάλαια καθώς ολοκληρώνονται.

1.3 Περιγραφή Δεδομένων

1.3.1 CIC-IDS-2017 Dataset

Το CIC-IDS-2017 δημιουργήθηκε από το Canadian Institute for Cybersecurity (CIC) και κυκλοφορεί ως σύνολο από οκτώ αρχεία CSV, ένα για κάθε ημέρα και χρονική ζώνη της εβδομαδιαίας καταγραφής:

- `Monday-WorkingHours.pcap_ISCX.csv` - μόνο κανονική κίνηση.

- `Tuesday-WorkingHours.pcap_ISCX.csv` - Brute Force (FTP/SSH).
- `Wednesday-workingHours.pcap_ISCX.csv` - DoS (Hulk, GoldenEye, slowloris, Slowhttptest, Heartbleed).
- `Thursday-WorkingHours-Morning-WebAttacks.pcap_ISCX.csv` - Brute Force, XSS, SQL Injection.
- `Thursday-WorkingHours-Afternoon-Infiltration.pcap_ISCX.csv` - Infiltration.
- `Friday-WorkingHours-Morning.pcap_ISCX.csv` - Botnet.
- `Friday-WorkingHours-Afternoon-PortScan.pcap_ISCX.csv` - Port Scan.
- `Friday-WorkingHours-Afternoon-DDos.pcap_ISCX.csv` - DDoS.

Κάθε γραμμή αντιστοιχεί σε μία *δικτυακή ροή* (flow), δηλαδή μια ακολουθία πακέτων μεταξύ ενός ζεύγους IP/ports για ένα συγκεκριμένο πρωτόκολλο. Τα features έχουν εξαχθεί με το εργαλείο **CICFlowMeter** και περιλαμβάνουν:

- **Βασικά** χαρακτηριστικά: Destination Port, Flow Duration, Total Fwd/Bwd Packets.
- **Στατιστικά μήκους πακέτων**: mean, std, min, max σε forward και backward κατευθύνσεις.
- **Στατιστικά χρονικών διαστημάτων** (Inter-Arrival Times, IAT): Flow IAT Mean/Std/Max/Min, Fwd IAT, Bwd IAT.
- **Flags TCP**: FIN/SYN/RST/PSH/ACK/URG Flag Count.
- **Bulk και subflow** στατιστικά, χαρακτηριστικά active/idle χρόνου.

Η δεύτερη στήλη που αφορά την εργασία είναι το **Label** - η ετικέτα της κλάσης (BENIGN ή όνομα επίθεσης).

1.3.2 Συνοπτικά Μεγέθη

Μέγεθος	Τιμή
Πηγαία αρχεία CSV	8
Αρχικές γραμμές	2.830.743
Αρχικές στήλες	79 (+ Label)
Κλάσεις (συμπ. BENIGN)	15
Μετά από καθαρισμό (γραμμές)	2.572.640
Μετά από feature selection	47 features + Label
Cached Parquet snapshot	221,9 MB

Πίνακας 1.1: Βασικά μεγέθη του CIC-IDS-2017 πριν και μετά την προεπεξεργασία.

1.4 Διάρθρωση της Αναφοράς

Η υπόλοιπη αναφορά οργανώνεται ως εξής:

- **Κεφάλαιο 2 - Θεωρητικό Υπόβαθρο.** Περιγράφονται οι θεμελιώδεις έννοιες που χρησιμοποιούνται σε όλα τα ερωτήματα: preprocessing, class imbalance, Pearson correlation, multicollinearity, κ.α.
- **Κεφάλαιο 3 - Ερώτημα 1: Exploratory Data Analysis.** Λεπτομερής περιγραφή της μεθοδο-

λογίας και των αποτελεσμάτων του Q1.

- **Κεφάλαιο 4 - Ερώτημα 2: Ταξινόμηση.** Εκπαίδευση και αξιολόγηση μοντέλων (θα συμπληρωθεί).
- **Κεφάλαιο 5 - Ερώτημα 3: Συσταδοποίηση.** Unsupervised learning και εξωτερική αξιολόγηση (θα συμπληρωθεί).
- **Κεφάλαιο 6 - Συμπεράσματα και Μελλοντικές Κατευθύνσεις.**
- **Παράρτημα** με οδηγίες εγκατάστασης, εκτέλεσης και αναπαραγωγής.

Κεφάλαιο 2

Θεωρητικό Υπόβαθρο

Σε αυτό το κεφάλαιο συνοψίζονται τα βασικά θεωρητικά εργαλεία που χρησιμοποιούνται στα τρία ερωτήματα. Οι έννοιες παρουσιάζονται όπως εμφανίζονται στο pipeline: από τον καθαρισμό δεδομένων (Q1) μέχρι την ταξινόμηση (Q2) και τη συσταδοποίηση (Q3).

2.1 Προεπεξεργασία Δεδομένων

2.1.1 Missing Values και Infinities

Ένα πραγματικό σύνολο δεδομένων σπάνια είναι πλήρες. Στις δικτυακές μετρήσεις του CIC-IDS-2017 οι *rate* στήλες (*Flow Bytes/s*, *Flow Packets/s*) υπολογίζονται ως διαιρέσεις με τη *Flow Duration*. Όταν η διάρκεια μιας ροής είναι μηδέν μικροδευτερόλεπτα (π.χ. *single-packet flow*), προκύπτουν τιμές $\pm\infty$. Η ανάγνωση αυτών από οποιονδήποτε *StandardScaler*, *RobustScaler* ή από τον ίδιο το *classifier* αποτυγχάνει.

Η τυπική πρακτική είναι:

1. Αντικατάσταση $\pm\infty$ με *NaN* (*df.replace*).
2. Επιλογή μεταξύ **drop rows** ή **imputation**:
 - **Drop**: απλό, διατηρεί την κατανομή, χάνει δεδομένα.
 - **Median imputation**: ανθεκτικό σε outliers, αλλά εισάγει bias σε *rate* στήλες.
 - **Zero imputation**: δηλώνει «μηδενικό throughput», συνήθως λάθος σε flow-based μετρίκες.

Όταν τα missing values είναι $< 1\%$ και σχετίζονται με ανώμαλες παρατηρήσεις (π.χ. μηδενική διάρκεια), η *drop-rows* είναι συνήθως η ασφαλέστερη επιλογή.

2.1.2 Ακριβή Διπλότυπα

Τα exact duplicates δημιουργούν δύο προβλήματα:

- **Πληθωρισμό κλάσεων**: αν χιλιάδες BENIGN γραμμές είναι πανομοιότυπες, το accuracy υπερεκτιμά τη γενικευτική ικανότητα του μοντέλου.
- **Data leakage μεταξύ train/test**: αν μια γραμμή εμφανιστεί και στα δύο split, το μοντέλο «απομνημονεύει» αντί να «μαθαίνει».

Γι' αυτό η αφαίρεση duplicates γίνεται **πριν** από κάθε split και όχι αφού.

2.2 Κλάσεις σε Ανισορροπία

Όταν η αναλογία της μεγαλύτερης προς τη μικρότερη κλάση ξεπερνά το $\sim 10 : 1$, μιλάμε για **imbalanced classification**. Στο CIC-IDS-2017 αυτή η αναλογία είναι $\sim 195\,000 : 1$ (BENIGN vs. Heartbleed), με τρεις άμεσες συνέπειες:

1. **Παραπλανητική accuracy.** Ένας ταξινομητής «always BENIGN» επιτυγχάνει $\sim 83\%$ χωρίς να ανιχνεύσει ούτε μία επίθεση. Γι' αυτό αναφέρουμε **macro-F1** και **per-class precision/recall**.
2. **Προκατάληψη εκπαίδευσης.** Οι default αλγόριθμοι ελαχιστοποιούν το total loss, άρα ευνοούν τη majority class. Αντίμετρα:
 - **class_weight='balanced'**: αντίστροφα βάρη στο loss.
 - **SMOTE** (Synthetic Minority Over-sampling Technique): synthetic samples για τις minority classes.
 - **Undersampling** της majority class.
3. **Stratified splitting υποχρεωτικό.** Ένα τυχαίο split μπορεί να στείλει όλες τις 11 Heartbleed γραμμές στο training set, αφήνοντας το test σύνολο χωρίς δείγματα για αξιολόγηση. Το **StratifiedShuffleSplit** διατηρεί τα αναλογικά ποσοστά ανά κλάση.

2.3 Pearson Correlation

Ο **συντελεστής συσχέτισης Pearson** μεταξύ δύο μεταβλητών X, Y ορίζεται ως:

$$r_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (2.1)$$

με $r \in [-1, +1]$. Τιμές ± 1 δηλώνουν τέλεια γραμμική εξάρτηση, $r \approx 0$ καμία γραμμική εξάρτηση (αλλά μπορεί να υπάρχει μη-γραμμική).

2.3.1 Τυφλά Σημεία

Η Pearson έχει τρεις γνωστούς περιορισμούς που είναι χρήσιμοι στην ερμηνεία των αποτελεσμάτων:

- Δεν ανιχνεύει **μη-γραμμικές** σχέσεις (π.χ. $Y = X^2$ δίνει $r \approx 0$).
- Είναι ιδιαίτερα ευαίσθητη σε **outliers**, ειδικά σε κατανομές heavy-tailed.
- Απαιτεί **αριθμητικές** μεταβλητές. Το label encoding που εφαρμόζουμε στο **Label** επιβάλλει αυθαίρετη διάταξη, οπότε η **corrwith(Label)** είναι μόνο *ενδεικτική*. Για αυστηρή αξιολόγηση feature importance χρησιμοποιείται **mutual information** ή tree-based importance.

2.4 Multicollinearity

Δύο features με $|r| \rightarrow 1$ μεταφέρουν την ίδια πληροφορία. Η **πολυσυγγραμμικότητα** (multicollinearity) βλάπτει τα μοντέλα με τρεις τρόπους:

1. **Γραμμικά μοντέλα** (Logistic Regression): οι συντελεστές γίνονται ασταθείς. Με δύο σχεδόν πανομοιότυπα features, ο αλγόριθμος μπορεί να δώσει $w_1 = +10$, $w_2 = -10$ ή $w_1 = +5$, $w_2 = -5$, μαθηματικά ισοδύναμα αλλά ανερμήνευτα. Η τυπική απόκλιση των coefficients εκρήγνυται.
2. **Αλγόριθμοι βασισμένοι σε αποστάσεις** (K-Means, KNN, DBSCAN): η επανειλημμένη διάσταση μετράει διπλά στην ευκλείδεια απόσταση, οπότε αυτή η διάσταση κυριαρχεί τεχνητά.
3. **Υπολογιστικά**: περισσότερες στήλες σημαίνει πιο αργή εκπαίδευση χωρίς κέρδος πληροφορίας.

Η στρατηγική που ακολουθούμε είναι να κρατήσουμε μία μόνο στήλη ανά cluster υψηλής συσχέτισης (με threshold $|r| > 0.95$), διατηρώντας την πρώτη στη σειρά των στηλών και πετώντας τις άλλες.

2.5 log1p Transform και Skewness

Πολλά features του CIC-IDS-2017 εκτείνονται σε πολλές τάξεις μεγέθους (π.χ. **Flow Duration** από μς έως λεπτά) και έχουν skewness > 100 με kurtosis στις χιλιάδες. Χωρίς μετασχηματισμό, τα ιστογράμματα συμπύσσονται σε μία κορυφή κοντά στο 0 με λεπτή ουρά και οι boxplots είναι οπτικά άχρηστα.

Ο μετασχηματισμός **log1p**:

$$\text{log1p}(x) = \log(1 + x) \quad (2.2)$$

έχει τρία πλεονεκτήματα:

- Είναι ασφαλής για $x = 0$ (ενώ $\log(0) \rightarrow -\infty$).
- Για μεγάλα x , $\text{log1p}(x) \approx \log(x)$.
- Για μικρά x , η Taylor expansion δίνει $\text{log1p}(x) \approx x$.

2.6 Stratified Sampling για Οπτικοποίηση

Για ένα dataset με ~ 2.57 εκατομμύρια γραμμές, το KDE και τα heatmaps γίνονται απαγορευτικά αργά. Η στρατηγική μας είναι:

- **Στατιστικά** (describe, correlation matrix): ολόκληρο το dataset. Δεν υπάρχει λόγος να θυσιάσουμε ακρίβεια.
- **Οπτικοποιήσεις**: stratified sample 50 000 γραμμών με `StratifiedShuffleSplit`, ώστε κάθε κλάση (ακόμα και η Heartbleed με 11 γραμμές) να εκπροσωπείται.

2.7 Αξιολόγηση Ταξινομητών (Q2)

2.7.1 Confusion Matrix

Για multi-class classification με C κλάσεις, η confusion matrix $M \in \mathbb{N}^{C \times C}$ ορίζεται ως M_{ij} = πλήθος δειγμάτων με πραγματική κλάση i που ταξινομήθηκαν στην κλάση j . Η διαγώνιος είναι τα σωστά (TP ανά κλάση).

2.7.2 Precision, Recall, F1

Για κάθε κλάση c :

$$\text{Precision}_c = \frac{TP_c}{TP_c + FP_c}, \quad (2.3)$$

$$\text{Recall}_c = \frac{TP_c}{TP_c + FN_c}, \quad (2.4)$$

$$F1_c = 2 \cdot \frac{\text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}. \quad (2.5)$$

Το **macro-F1** είναι ο αστάθμιστος μέσος όρος των $F1_c$ - κάθε κλάση μετράει το ίδιο, ανεξάρτητα από το μέγεθός της. Το **weighted-F1** σταθμίζει με το support, επομένως υπερεκτιμά τα μοντέλα που τα πάνε καλά στη majority class.

2.7.3 Grid Search Cross-Validation

Το `GridSearchCV` εξαντλητικά δοκιμάζει όλους τους συνδυασμούς hyperparameters σε k -fold cross-validation (συνήθως $k = 5$). Η **stratified** παραλλαγή διατηρεί την κατανομή των κλάσεων σε κάθε fold, κρίσιμο για imbalanced data.

2.8 Συσταδοποίηση (Q3)

2.8.1 K-Means

Ο αλγόριθμος K-Means ελαχιστοποιεί την εντός-συστάδας διασπορά (inertia):

$$\min_{\{C_k\}, \{\mu_k\}} \sum_{k=1}^K \sum_{x \in C_k} \|x - \mu_k\|^2. \quad (2.6)$$

Επιλογή του K : elbow method (inertia vs K) ή silhouette score.

2.8.2 DBSCAN

Σε αντίθεση με το K-Means, το DBSCAN δεν απαιτεί προκαθορισμένο K , βρίσκει συστάδες αυθαίρετου σχήματος και χαρακτηρίζει σημεία ως **noise** αν δεν ανήκουν σε κάποια πυκνή περιοχή. Ορίζεται από δύο παραμέτρους:

- ϵ (eps): ακτίνα γειτονιάς.
- `min_samples`: ελάχιστα σημεία σε μια πυκνή περιοχή.

2.8.3 Hierarchical Clustering

Κατασκευάζει ένα δενδρόγραμμα (dendrogram) συγχωνεύοντας επαναληπτικά τα πιο κοντινά σημεία-/συστάδες. Η linkage function (single, complete, average, Ward) καθορίζει πώς μετράται η απόσταση μεταξύ συστάδων.

2.8.4 External Validation με ARI

Όταν οι *πραγματικές* ετικέτες είναι γνωστές (όπως εδώ), μπορούμε να αξιολογήσουμε τις συστάδες με **Adjusted Rand Index (ARI)** ή **Normalized Mutual Information (NMI)**. Αυτό μας λέει πόσο καλά η δομή των δεδομένων αντιστοιχεί στις κλάσεις επίθεσης, χωρίς να έχει χρησιμοποιηθεί καμία ετικέτα κατά την εκπαίδευση.

Κεφάλαιο 3

Ερώτημα 1: Exploratory Data Analysis

3.1 Επισκόπηση του Pipeline

Το Q1 υλοποιήθηκε ως ένα μοναδικό Jupyter notebook (`notebooks/q1_eda.ipynb`), εύκολα επανεκτελέσιμο μέσω `nbconvert`. Το pipeline που ακολουθεί είναι αυτό:

1. Φόρτωση όλων των 8 CSV αρχείων και συγχώνευσή τους.
2. Κανονικοποίηση ονομάτων στηλών και ετικετών.
3. Αντιμετώπιση Inf και NaN.
4. Αφαίρεση duplicates.
5. Περιγραφικά στατιστικά και οπτικοποίηση κατανομής κλάσεων.
6. Ιστογράμματα, boxplots, correlation heatmap.
7. Feature selection (zero-variance + high-correlation).
8. Αποθήκευση σε Parquet για επαναχρησιμοποίηση από Q2/Q3.

3.2 Φόρτωση και Επισκόπηση

3.2.1 Ενοποίηση των 8 CSV

Η βοηθητική συνάρτηση `load_cic_ids_2017` (αρχείο `utils/helpers.py`) διαβάζει ένα προς ένα τα 8 CSV αρχεία με `encoding='latin1'` για να αντιμετωπίσει τα corrupted bytes στα labels των Web Attacks, κάνει concatenation και κρατά προέλευση μέσω της στήλης `__source_file` (αφαιρείται αργότερα).

```
● ● ● Φόρτωση Dataset
1 from utils.helpers import load_cic_ids_2017, RANDOM_STATE
2 import numpy as np, pandas as pd
3
4 df = load_cic_ids_2017()
5 print(df.shape) # (2830743, 80)
```

Μέσα στη `_standardize_columns` γίνεται και ένα κρίσιμο βήμα: η στήλη `Fwd Header Length.1` - ένα αντίγραφο της `Fwd Header Length` που εξάγει λανθασμένα το CICFlowMeter - αφαιρείται αυτόματα.

3.2.2 Καθαρισμός Ετικετών

Οι ετικέτες των Web Attacks στα raw CSV περιέχουν το χαρακτήρα `U+FFFD` (το Unicode replacement character) στη θέση ενός en-dash: "Web Attack `□` Brute Force". Η `_clean_labels` τα κα-

νονικοποιεί σε "Web Attack Brute Force", "Web Attack XSS", "Web Attack Sql Injection".

3.3 Καθαρισμός Δεδομένων

3.3.1 Infinities και Missing Values

Οι στήλες Flow Bytes/s και Flow Packets/s περιέχουν 4 376 τιμές $\pm\infty$ (από διαίρεση με Flow Duration=0). Ακολουθούμε τη στρατηγική replace-&-drop:

```

1 from utils.helpers import handle_infinities
2
3 df = handle_infinities(df) # (+/-inf) -> NaN
4 print(df.isna().sum().sum()) # 4376 NaN total
5 before = len(df)
6 df = df.dropna().reset_index(drop=True)
7 print(before - len(df), "rows dropped") # 2867 rows

```

Το αποτέλεσμα είναι **2.867** γραμμές που αφαιρέθηκαν, δηλαδή 0.101% του dataset - ποσοστό πολύ μικρό για να επηρεάσει τη δομή.

3.3.2 Duplicates

Η `drop_duplicates_report` αναφέρει **255.236** exact duplicates ($\sim 9\%$ μετά το NaN handling). Αυτά αφαιρούνται *πριν* το οποιοδήποτε split, ώστε το test set του Q2 να είναι πραγματικά unseen.

Στάδιο	Γραμμές	Διαφορά
Αρχικό	2.830.743	—
Μετά drop NaN	2.827.876	−2 867
Μετά drop dup	2.572.640	−255 236

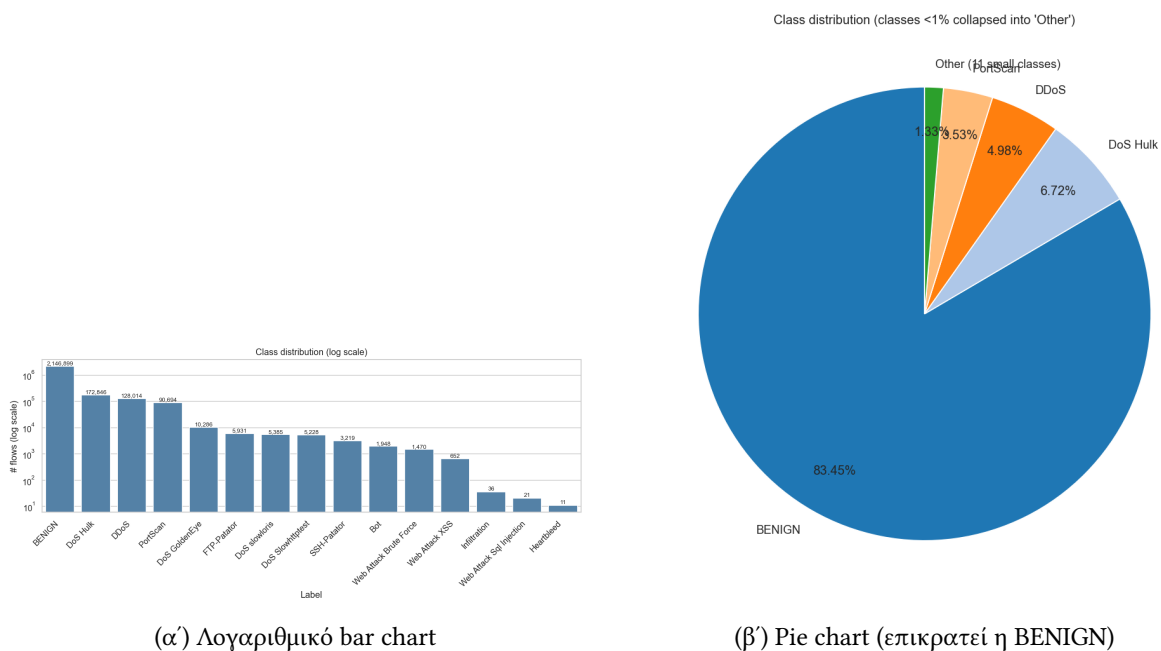
Πίνακας 3.1: Προοδευτική μείωση του μεγέθους του dataset κατά τον καθαρισμό.

3.4 Κατανομή Κλάσεων

Η κατανομή των 15 κλάσεων φαίνεται στον Πίνακα 3.2 και οπτικοποιείται στο Σχήμα 3.1.

Κλάση	Πλήθος	Ποσοστό
BENIGN	2.146.899	83,45%
DoS Hulk	172.846	6,72%
DDoS	128.014	4,98%
PortScan	90.694	3,53%
DoS GoldenEye	10.286	0,40%
FTP-Patator	5.931	0,23%
DoS slowloris	5.385	0,21%
DoS Slowhttptest	5.228	0,20%
SSH-Patator	3.219	0,13%
Bot	1.948	0,08%
Web Attack Brute Force	1.470	0,06%
Web Attack XSS	652	0,03%
Infiltration	36	0,0014%
Web Attack Sql Injection	21	0,0008%
Heartbleed	11	0,0004%
Σύνολο	2.572.640	100%

Πίνακας 3.2: Κατανομή των 15 κλάσεων μετά τον καθαρισμό. Το imbalance ratio (max/min) είναι $\approx 195\,173\times$.



Σχήμα 3.1: Κατανομή κλάσεων στο καθαρισμένο CIC-IDS-2017. Το αριστερό σχήμα χρησιμοποιεί λογαριθμικό άξονα ώστε να φαίνονται και οι σπάνιες κλάσεις (Heartbleed, Sql Injection, Infiltration).

Παρατηρήσεις. Η ακραία ανισορροπία (η BENIGN είναι $> 195\,000\times$ πιο συχνή από την Heartbleed) καθορίζει τις επιλογές του Q2: θα απαιτηθεί `class_weight="balanced"`, macro-F1 ως κύρια metric και stratified splitting.

3.5 Περιγραφικά Στατιστικά

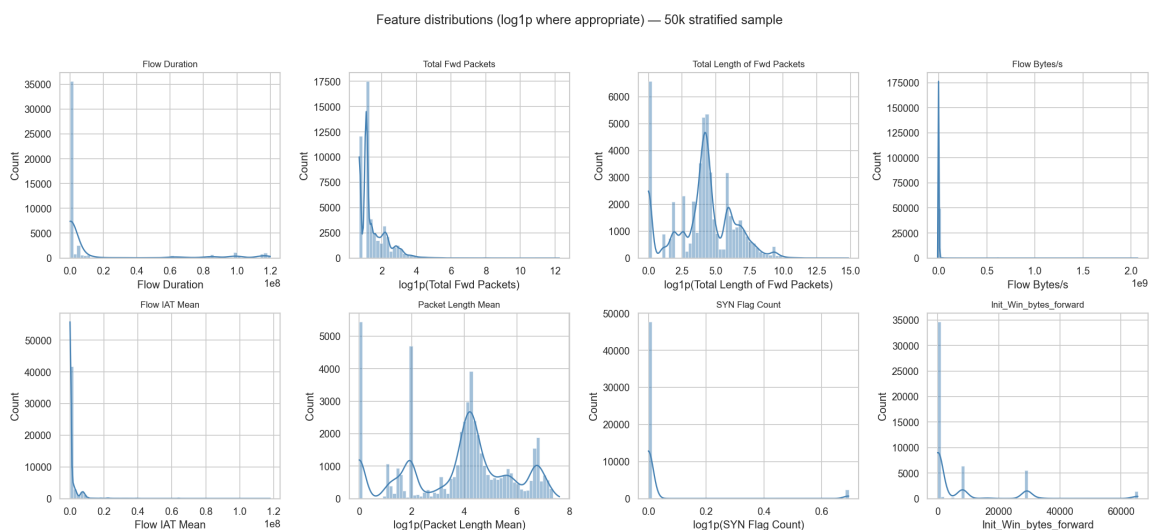
Για κάθε αριθμητική στήλη υπολογίστηκαν mean, median, std, min, 25%, 50%, 75%, max, skewness και kurtosis. Τα πλήρη αποτελέσματα αποθηκεύτηκαν στο `outputs/results/q1_describe.csv`. Τα βασικά ευρήματα:

- Πολλές στήλες έχουν **skewness** > 100 και **kurtosis** στις χιλιάδες - επιβεβαιώνοντας κατανομές heavy-tailed που απαιτούν log-transform για οπτικοποίηση.
- Οι στήλες `Fwd Avg Bytes/Bulk`, `Bwd Avg Packets/Bulk` κ.ά. έχουν **std=0**, δηλαδή είναι **σταθερές** σε όλες τις $\sim 2.57\text{M}$ γραμμές και θα αφαιρεθούν στο βήμα feature selection.

3.6 Οπτικοποιήσεις

3.6.1 Ιστογράμματα Top Features

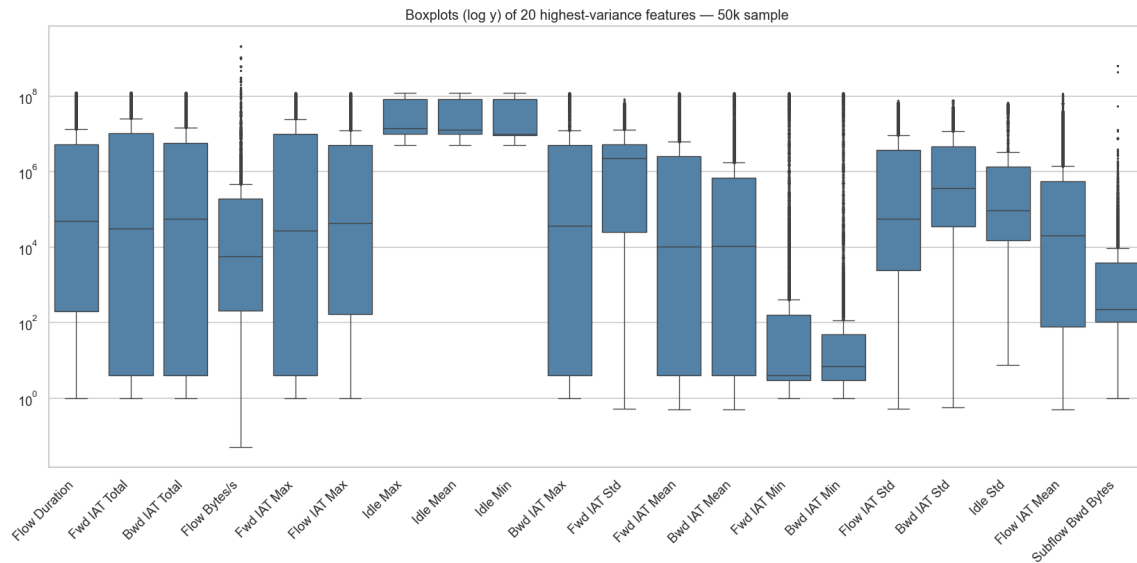
Επιλέχθηκαν 8 features με τη μεγαλύτερη διασπορά για plot. Για λόγους ευκρίνειας ο άξονας x είναι σε log scale ($\log_{10}$) και το KDE υπολογίστηκε σε stratified sample 50 000 γραμμών.



Σχήμα 3.2: Ιστογράμματα με KDE για τις 20 στήλες μεγαλύτερης διασποράς ($\log_{10}$ scale). Οι περισσότερες κατανομές είναι έντονα λοξές και heavy-tailed.

3.6.2 Boxplots

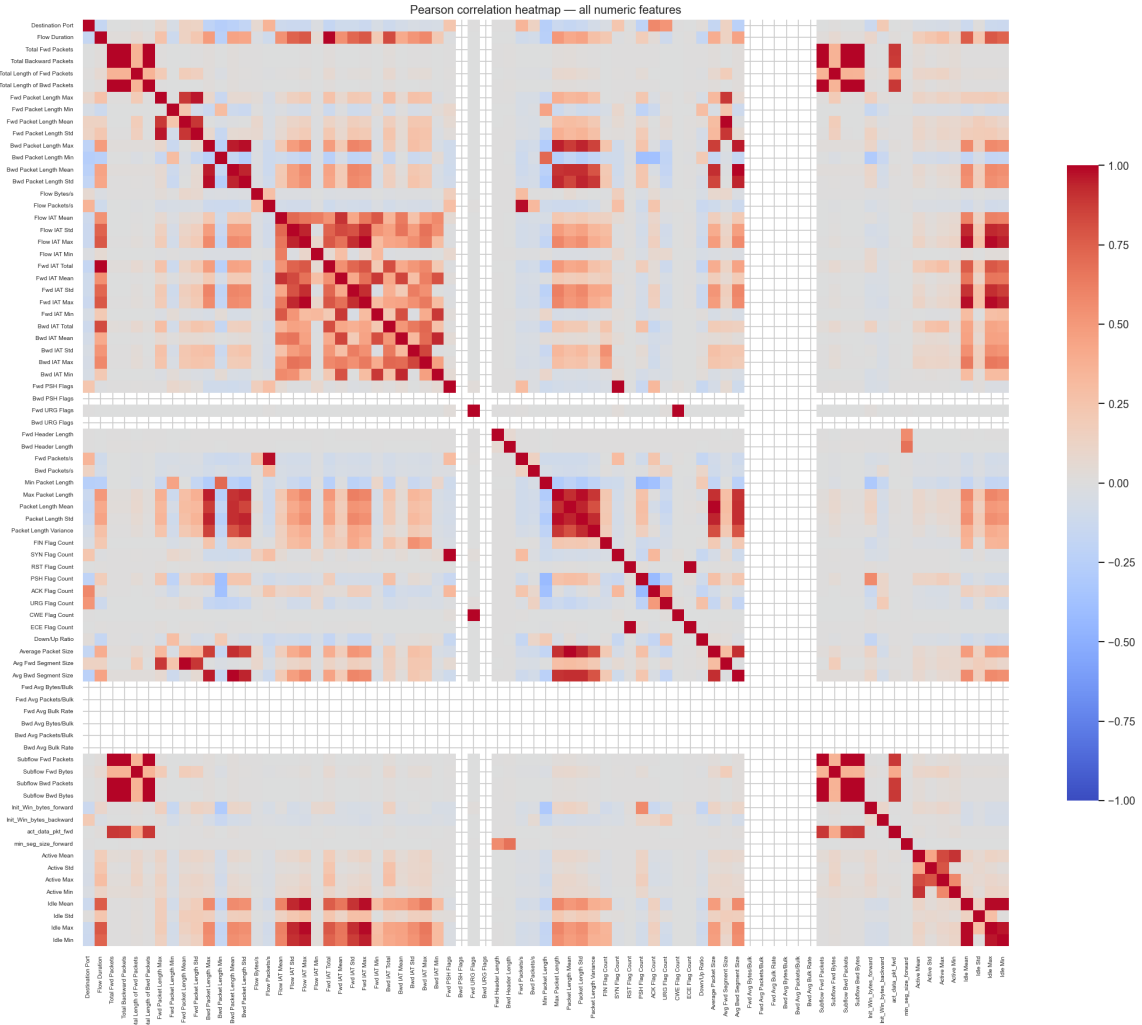
Οι boxplots (Σχήμα 3.3) αποκαλύπτουν έντονα «νέφοι» outliers στις rate και duration στήλες. Αυτά δεν είναι «θόρυβος» προς αφαίρεση - είναι ακριβώς η υπογραφή των DDoS/DoS επιθέσεων (πολύ υψηλά throughput, πολύ μακριές διάρκειες).



Σχήμα 3.3: Boxplots (log_{1p} scale) για τις 20 στήλες μεγαλύτερης διασποράς. Τα outliers πάνω από τα whiskers είναι διαγνωστικά για επιθέσεις DoS/DDoS και δεν αφαιρούνται.

3.6.3 Correlation Heatmap

Η Pearson correlation matrix (77×77) οπτικοποιείται ως heatmap (Σχήμα 3.4). Τα βαθυκόκκινα και βαθυγάλανα τετράγωνα είναι τα ζεύγη που θα στοχεύσουμε στο feature selection.



Σχήμα 3.4: Pearson correlation heatmap των 77 αριθμητικών features. Τα μεγάλα κόκκινα clusters αντιστοιχούν σε αλγοριθμικές ταυτότητες του CICFlowMeter (π.χ. Subflow Fwd Bytes $\approx$ Total Length of Fwd Packets).

3.6.4 Top Correlated Pairs

Τα 15 ζεύγη με τη μεγαλύτερη $|r|$ φαίνονται στον Πίνακα 3.3. Τα επτά πρώτα ζεύγη έχουν τέλεια συσχέτιση ($r = 1.0$) - δηλαδή είναι **αλγοριθμικές ταυτότητες** που ο CICFlowMeter εξαγεί δύο φορές με διαφορετικά ονόματα.

Feature A	Feature B	r
Bwd Packet Length Mean	Avg Bwd Segment Size	1,0000
Total Fwd Packets	Subflow Fwd Packets	1,0000
Total Backward Packets	Subflow Bwd Packets	1,0000
Fwd URG Flags	CWE Flag Count	1,0000
Fwd Packet Length Mean	Avg Fwd Segment Size	1,0000
Fwd PSH Flags	SYN Flag Count	1,0000
Total Length of Bwd Packets	Subflow Bwd Bytes	1,0000
Total Length of Fwd Packets	Subflow Fwd Bytes	1,0000
Total Backward Packets	Subflow Fwd Packets	0,9991
Subflow Fwd Packets	Subflow Bwd Packets	0,9991
Total Fwd Packets	Subflow Bwd Packets	0,9991
Total Fwd Packets	Total Backward Packets	0,9991
Flow Duration	Fwd IAT Total	0,9985
Flow IAT Max	Fwd IAT Max	0,9981
Packet Length Mean	Average Packet Size	0,9978

Πίνακας 3.3: Top-15 ζεύγη με $|r| > 0.997$. Τα 8 πρώτα είναι αλγοριθμικές ταυτότητες του CICFlowMeter.

3.6.5 Correlation με το Label

Καμία στήλη δεν έχει $|r| > 0.31$ με το label (εξαρτώμενο από label encoding), επιβεβαιώνοντας ότι το πρόβλημα είναι γνήσια πολυμεταβλητό. Οι 5 πιο σχετικές στήλες είναι **Fwd IAT Std** (0,30), **Idle Mean** (0,28), **Bwd Packet Length Std** (0,28), **Idle Max** (0,28), **Idle Min** (0,28).

Σημείωση: Τέσσερα από τα παραπάνω πέντε features (**Idle Mean/Max/Min**, **Bwd Packet Length Std**) θα αφαιρεθούν στην επόμενη ενότητα ως υψηλά συσχετισμένα. Αυτό δεν αφαιρεί πληροφορία: ανήκουν στο ίδιο cluster με features που διατηρούνται (π.χ. **Flow IAT Max** με $r = 0,276$ προς το label, που εκπροσωπεί τη ομάδα **Idle/IAT-Max**). Έτσι ο επόμενος ταξινομητής βλέπει την ίδια διακριτική σήμανση μέσω του εκπροσώπου της κάθε ομάδας.

3.7 Feature Selection

3.7.1 Κριτήρια Αφαίρεσης

Εφαρμόσαμε τρία κριτήρια, με την ακόλουθη σειρά:

Κριτήριο	Αιτιολογία
Bookkeeping	Βοηθητικές στήλες (<code>__source_file</code>) που δεν είναι feature.
Zero variance ($\sigma^2 = 0$)	Σταθερές στήλες: καμία πληροφορία, καταρρίπτουν κάθε scaler/model.
High correlation ($ r > 0.95$)	Multicollinear near-duplicates: κρατάμε μία ανά cluster.

Πίνακας 3.4: Τα τρία κριτήρια feature selection και η αιτιολογία τους.

3.7.2 Αποτελέσματα

Συνολικά αφαιρέθηκαν **31 στήλες** (Πίνακας 3.5):

- **1** bookkeeping (`__source_file`).
- **8** zero-variance: όλες οι Bulk-rate (Fwd/Bwd Avg Bytes/Bulk, .../Packets/Bulk, .../Bulk Rate) και Bwd PSH Flags, Bwd URG Flags. Όλες είναι μηδενικές σε όλα τα $\sim 2.57\text{M}$ flows.
- **22** highly correlated: αλγοριθμικές ταυτότητες του CICFlowMeter και near-duplicates (Average Packet Size, Avg Bwd Segment Size, Subflow Fwd Packets, κλπ.).

Στήλη	Λόγος
<code>__source_file</code>	bookkeeping / non-feature helper column
Bwd PSH Flags	zero variance (constant)
Bwd URG Flags	zero variance (constant)
Fwd Avg Bytes/Bulk	zero variance (constant)
Fwd Avg Packets/Bulk	zero variance (constant)
Fwd Avg Bulk Rate	zero variance (constant)
Bwd Avg Bytes/Bulk	zero variance (constant)
Bwd Avg Packets/Bulk	zero variance (constant)
Bwd Avg Bulk Rate	zero variance (constant)
Average Packet Size	highly correlated ($ r > 0.95$) with a kept feature
Avg Bwd Segment Size	highly correlated ($ r > 0.95$)
Avg Fwd Segment Size	highly correlated ($ r > 0.95$)
Bwd Packet Length Mean	highly correlated ($ r > 0.95$)
Bwd Packet Length Std	highly correlated ($ r > 0.95$)
CWE Flag Count	highly correlated ($ r > 0.95$)
ECE Flag Count	highly correlated ($ r > 0.95$)
Fwd IAT Max	highly correlated ($ r > 0.95$)
Fwd IAT Total	highly correlated ($ r > 0.95$)
Fwd Packet Length Std	highly correlated ($ r > 0.95$)
Fwd Packets/s	highly correlated ($ r > 0.95$)
Idle Max	highly correlated ($ r > 0.95$)
Idle Mean	highly correlated ($ r > 0.95$)
Idle Min	highly correlated ($ r > 0.95$)
Packet Length Std	highly correlated ($ r > 0.95$)
SYN Flag Count	highly correlated ($ r > 0.95$)
Subflow Bwd Bytes	highly correlated ($ r > 0.95$)
Subflow Bwd Packets	highly correlated ($ r > 0.95$)
Subflow Fwd Bytes	highly correlated ($ r > 0.95$)
Subflow Fwd Packets	highly correlated ($ r > 0.95$)
Total Backward Packets	highly correlated ($ r > 0.95$)
Total Length of Bwd Packets	highly correlated ($ r > 0.95$)

Πίνακας 3.5: Οι 31 στήλες που αφαιρέθηκαν στο feature selection.

3.7.3 Τελική Λίστα Features

Μετά το feature selection έχουμε **47 features + Label** (48 στήλες συνολικά), οι οποίες αποθηκεύτηκαν στο `outputs/results/q1_final_feature_list.csv`. Ενδεικτικά, οι πρώτες δέκα: Destination Port, Flow Duration, Total Fwd Packets, Total Length of Fwd Packets, Fwd Packet Length Max, Fwd Packet Length Min, Fwd Packet Length Mean, Bwd Packet Length Max, Bwd Packet Length Min, Flow Bytes/s.

3.8 Αποθήκευση του Clean Snapshot

Το καθαρισμένο dataset σώζεται σε Parquet (`data/cic_ids_2017_clean.parquet`, 221,9 MB) μέσω της `save_clean_cached`. Τα Q2 και Q3 το φορτώνουν άμεσα με `load_clean_cached()` χωρίς να επαναλάβουν το pipeline.

Γιατί Parquet και όχι CSV:

- **Columnar storage:** αν το Q3 θέλει μόνο 10 στήλες, τις διαβάζει χωρίς να φορτώσει τις υπόλοιπες 38.
- **Compressed:** 221,9 MB αντί για ~900 MB σε CSV.
- **Type-preserved:** διατηρεί τα int32/float32 που προσεκτικά ορίσαμε στη `_optimize_memory`.
- **Fast I/O:** τα Q2/Q3 φορτώνουν σε < 2 sec αντί για ~ 60 sec CSV parsing.

3.9 Σύνοψη Q1

Μέγεθος	Τιμή
Αρχικές γραμμές	2.830.743
Rows με $\pm\infty \rightarrow \text{NaN}$	4.376 (τιμές)
Rows που έπεσαν λόγω NaN	2.867 (0, 101%)
Exact duplicates που αφαιρέθηκαν	255.236 ($\sim 9\%$)
Τελικές γραμμές	2.572.640
Αρχικές στήλες	79 + Label
Στήλες που αφαιρέθηκαν	31
Τελικές στήλες	47 + Label
Κλάσεις	15
Imbalance ratio (max/min)	$\sim 195\,173\times$
Parquet snapshot	221,9 MB

Πίνακας 3.6: Βασικά αποτελέσματα του Q1.

Κεφάλαιο 4

Ερώτημα 2: Ταξινόμηση

Στο κεφάλαιο αυτό εκπαιδεύουμε τρεις ταξινομητές (Logistic Regression, Decision Tree, Random Forest) στο καθαρισμένο dataset του Q1 και τους αξιολογούμε με δύο διαφορετικά πρωτόκολλα hyperparameter selection: (α) *Σενάριο A* - held-out validation set σε stratified split 60/20/20, και (β) *Σενάριο B* - 5-fold Stratified CV μέσω GridSearchCV πάνω στο 80% training, με το ίδιο test set σε όλες τις παραλλαγές.

4.1 Μεθοδολογία

4.1.1 Balanced Subsampling σε Target Μέγεθος

Το πλήρες cleaned dataset έχει 2.572.640 γραμμές. Αντί για flat ποσοστιαία μείωση, ορίζουμε **απευθείας το τελικό πλήθος rows** (TARGET_ROWS) και μια στρατηγική BALANCED_SAMPLING. Η συνάρτηση subsample_to_target (στο utils) υλοποιεί δύο σχήματα:

- **balanced** (BALANCED_SAMPLING=True - η επιλογή μας): οι τέσσερις μεγάλες κλάσεις (BENIGN, DoS, Hulk, DDoS, PortScan) μπαίνουν με *ίσο ceiling* (~ 75K η καθεμία στο 500K run), ενώ οι rare attacks (Heartbleed = 11, SQL Injection = 21, Infiltration = 36) κρατιούνται **πλήρεις**.
- **proportional** (=False): διατηρεί το πραγματικό prior (~ 83% BENIGN).

Γιατί balanced για classification. Το training set ενός classifier δεν χρειάζεται να αναπαριστά την πραγματικότητα - χρειάζεται να αναπαριστά τα *σύνορα μεταξύ των κλάσεων*. Επιβεβαιώσαμε ότι το proportional δείγμα έχει δύο σοβαρά προβλήματα: (α) *information starvation* (το Bot πέφτει 1.948 → 113, το Web Attack Brute Force 1.470 → 85 - πολύ λίγα rows για 47 features), και (β) *proportional inversion* (ο rare-floor κρατά το XSS=652 πλήρες αλλά κόβει το Brute Force σε 85, οπότε το XSS καταλήγει ~ 8× μεγαλύτερο - αντιστρέφει την πραγματική συχνότητα). Το balanced τα λύνει και τα δύο - σε συνδυασμό με `class_weight="balanced"` σε επίπεδο μοντέλου, δίνει και στις 15 κλάσεις πραγματική ευκαιρία να μαθευτούν. Η ποσοτική σύγκριση balanced vs proportional δίνεται στην ενότητα 4.5.

Τελικό μέγεθος (500K balanced run): 500.000 rows × 47 features + Label.

Configurable mode. Η TARGET_ROWS έχει πρακτικά presets: 150_000 (fast), 500_000 (το run που παρουσιάζουμε) και None (full ~ 2,5M). Σε μεγάλα samples ο κύριος bottleneck είναι το LR με `lbfgs` (~ 450s split / ~ 1700s CV στο 500K)· τα DT/RF επιταχύνονται με Intel `scikit-learn-intelex` (oneDAL) - βλ. Κεφάλαιο B'.

4.1.2 Encoding, Scaling και Imbalance Handling

- **LabelEncoder** στο target: μετατρέπει τα 15 string labels σε ακραίους [0, 14], όπως περιμένουν οι multi-class classifiers του scikit-learn. One-hot δεν έχει νόημα στο target (θα γινόταν multi-output regression).
- **StandardScaler μέσα σε Pipeline** για το Logistic Regression. Το pipeline εξασφαλίζει ότι ο scaler γίνεται `fit` μόνο στο training fold σε κάθε CV split - αποφεύγοντας data leakage.

Για Decision Tree και Random Forest παραλείπεται: τα splits του δέντρου είναι invariant σε monotonic transforms του feature.

- **class_weight="balanced"** σε κάθε αλγόριθμο. Ζυγίζει κάθε row με $n/(C \cdot n_c)$ όπου n το σύνολο, C ο αριθμός κλάσεων και n_c η συχνότητα της κλάσης. Χωρίς αυτό, ένας classifier θα έβγαζε $\sim 80\%$ accuracy απλώς προβλέποντας πάντα **BENIGN**. Εναλλακτικές (SMOTE, undersampling) εγκυμονούν data-leakage risks αν δεν εφαρμοστούν per-fold.

4.1.3 Πρωτόκολλο Splits

Το subsampled dataset σπάει σε δύο βήματα:

1. Stratified 80/20 $\rightarrow$ **trainval** / **test**. Το **test** (100.000 rows στο 500K run) είναι ο **τυφλός μάρτυρας**: αγγίζεται μόνο για το final reporting και των δύο σεναρίων.
 2. Από το **trainval** σπάμε ξανά stratified 75/25 $\rightarrow$ **train** (300.000 rows) / **val** (100.000 rows). Αυτό δίνει τον τελικό 60/20/20 λόγο που ζητά η εκφώνηση.
- **Σενάριο A** (split-based grid search): για κάθε combination στο grid, fit σε **train**, score σε **val** με F1-weighted. Επιλέγουμε best params, refit στο **trainval**, evaluate στο **test**.
 - **Σενάριο B** (5-fold StratifiedKFold μέσω GridSearchCV) πάνω στο ίδιο **trainval** και ίδιο grid. Best params $\rightarrow$ refit στο **trainval**, evaluate στο **test**.

Το κοινό **test** σε κάθε παραλλαγή είναι αυτό που καθιστά δυνατή τη μεταξύ τους σύγκριση.

4.1.4 Scoring Metric για την Επιλογή Hyperparameters

Χρησιμοποιούμε **F1-weighted** και στα δύο σενάρια. Στο final reporting εμφανίζουμε και F1-macro. Λόγοι:

- **Accuracy** εξαπατά σε imbalanced multi-class ($\sim 80\%$ απλώς από το να προβλέπει BENIGN πάντα).
- **F1-macro** δίνει ίσο βάρος σε κάθε κλάση, αλλά οι rare κλάσεις (Heartbleed = 11 rows) έχουν τέτοια variance ώστε να ταλαντώνεται έντονα κατά τη διαδικασία selection.
- **F1-weighted** ζυγίζει κάθε κλάση με τη support της - είναι default για multi-class στο scikit-learn, και σταθερότερη μετρική για hyperparameter selection.

Το F1-macro κρατιέται ως *μετρική αναφοράς* για να κρίνουμε αν το μοντέλο απλώς «μαθαίνει τις μεγάλες κλάσεις» ή αν πιάνει και τις rare attacks.

4.2 Logistic Regression

Διαίσθηση. Γραμμικός classifier, softmax για multi-class: μία γραμμική λειτουργία ανά κλάση, normalized να αθροίζουν σε 1. Η απόφαση χωρίζεται από *υπερεπίπεδα* στο feature space. Καλός baseline: ταχύτατο, ερμηνεύσιμο (coefficients), δίνει calibrated probabilities. Περιορισμοί: υποθέτει γραμμική σχέση features $\rightarrow$ log-odds, ευαίσθητο σε scale, δεν πιάνει αλληλεπιδράσεις features.

Grid. 8 combinations:

$$C \in \{0.01, 0.1, 1, 10\}, \quad \text{solver} = \text{lbfgs}, \quad \text{max_iter} \in \{200, 500\}.$$

Ο **saga** αφαιρέθηκε από το grid: σε δοκιμές βγήκε σταθερά 7ος-9ος (ποτέ δεν επιλέχθηκε) και ήταν $\sim 6\times$ πιο αργός· ο **newton-cholesky** ήταν $\sim 10\times$ πιο αργός και προκαλούσε εξάντληση μνήμης. Λεπτομέρειες στο Κεφάλαιο **B'**.

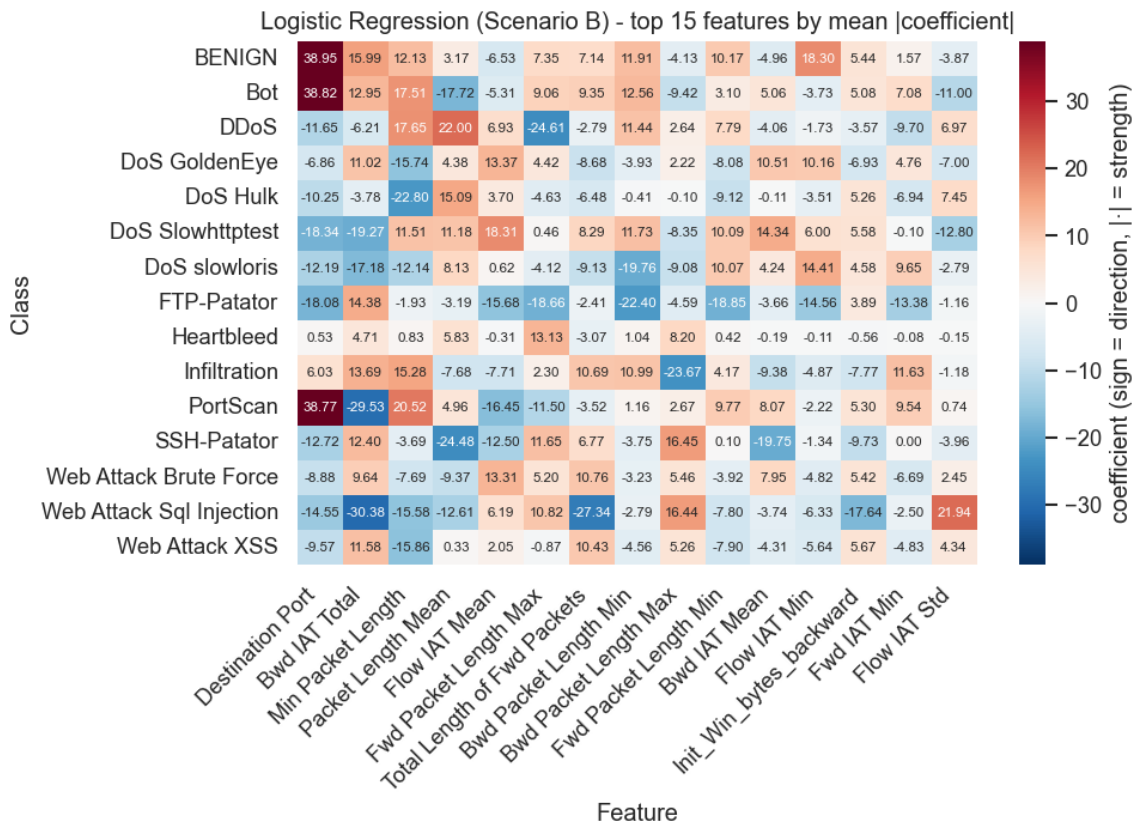
Αποτέλεσμα. Και στα δύο σενάρια επιλέγονται τα ίδια hyperparameters: $C = 10$, `solver=lbfgs`, `max_iter = 500`. Η σύγκλιση των δύο σεναρίων δείχνει ότι το validation set των 100.000 rows είναι αρκετά μεγάλο ώστε να συμφωνήσει με τον CV μέσο όρο. Το metrics όπως το βλέπουμε στο test:

Σενάριο	Accuracy	Precision _w	Recall _w	F1 _w	F1 _m
A (split)	0,9426	0,9761	0,9426	0,9548	0,6703
B (CV)	0,9426	0,9761	0,9426	0,9548	0,6703

Πίνακας 4.1: Logistic Regression - test metrics (500K balanced).

Το F1-weighted = 0,955 ακούγεται εξαιρετικό αλλά το F1-macro = 0,670 αποκαλύπτει την αδυναμία: το LR αποτυγχάνει στις non-linearly separable κλάσεις. (Με balanced sampling το macro ανέβηκε αισθητά σε σχέση με proportional – βλ. 4.5.) Όπως αναμέναμε, γραμμικά υπερεπίπεδα δεν αρκούν για να ξεχωρίσουν Web-Attack υπογραφές μέσα σε BENIGN HTTP traffic.

Συντελεστές του LR. Το multinomial LR παράγει έναν πίνακα συντελεστών shape $(n_{\text{classes}}, n_{\text{features}})$. Κάθε row δείχνει πόσο σπρώχνει κάθε feature ένα sample προς (θετικό πρόσημο) ή μακριά από (αρνητικό) την κλάση. Σε αντίθεση με τα tree-based feature importances, οι LR συντελεστές δίνουν *direction*, όχι μόνο magnitude - αυτό είναι το ερμηνεύσιμο πλεονέκτημα του LR.



Σχήμα 4.1: Logistic Regression (Σενάριο B) - heatmap των συντελεστών για τα top-15 features (κατά μέσο |coef| σε όλες τις κλάσεις). Έντονο κόκκινο = το feature οδηγεί strongly προς αυτή την κλάση· έντονο μπλε = αντενδείκνυται. Παράδειγμα: η PortScan έχει σαφώς θετικούς συντελεστές στα Bwd Packet Length Min και Bwd Packets/s (μικρά backward packets σε υψηλό rate - υπογραφή scanning).

4.3 Decision Tree

Διαίσθηση. Ακολουθία `if/else` ερωτήσεων· σε κάθε κόμβο επιλέγουμε `feature+threshold` που μεγιστοποιεί το *information gain* (μείωση impurity). Τα criteria που συγκρίνουμε:

- **Gini impurity:** $1 - \sum_c p_c^2$. Πιθανότητα λανθασμένης ταξινόμησης αν βάλουμε τυχαία row στο πιο συχνό class.
- **Entropy:** $-\sum_c p_c \log_2 p_c$. Information- theoretic measure - πόσα bits χρειάζονται για να περιγράψουμε την κλάση.

Regularization μέσω `max_depth` (upper bound) και `min_samples_split` (lower bound μεγέθους κόμβου). Δεν απαιτείται scaling.

Grid. 24 combinations:

$$\text{max_depth} \in \{5, 10, 20, \text{None}\}, \text{min_samples_split} \in \{2, 10, 50\}, \text{criterion} \in \{\text{gini}, \text{entropy}\}.$$

Αποτέλεσμα. Σενάριο A επιλέγει `max_depth = None` (unbounded), `min_samples_split = 2`, `entropy`. Σενάριο B επιλέγει `max_depth = 20`, `min_samples_split = 2`, `entropy`. Η διαφορά είναι αμελητέα σε μετρικές απόδοσης:

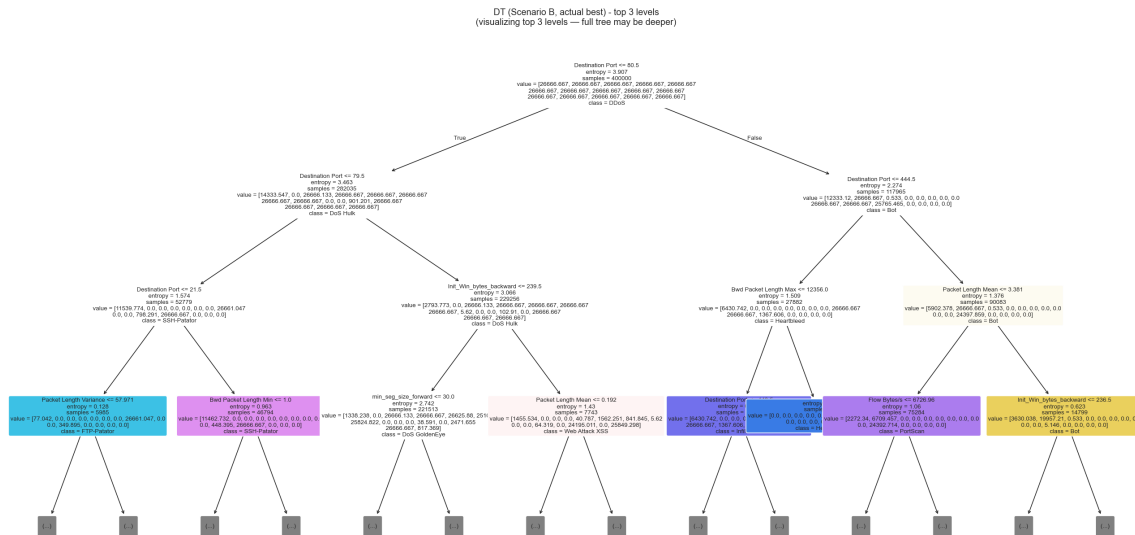
Σενάριο	Accuracy	Precision _w	Recall _w	F1 _w	F1 _m
A (split)	0,9973	0,9973	0,9973	0,9973	0,8705
B (CV)	0,9971	0,9973	0,9971	0,9971	0,8691

Πίνακας 4.2: Decision Tree - test metrics (500K balanced).

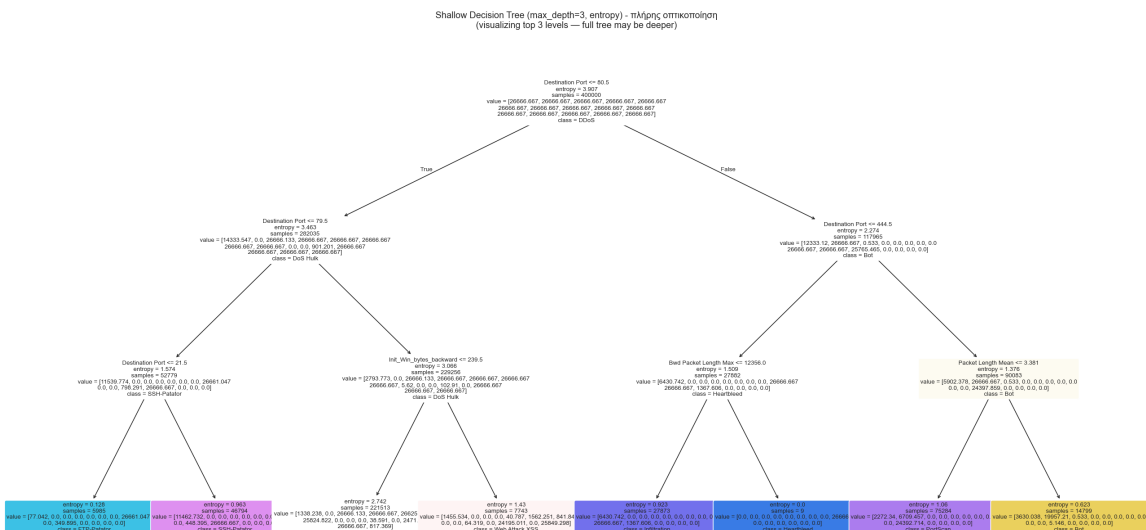
Το F1-macro βελτιώνεται κατά ~ 13 ποσοστιαίες μονάδες σε σχέση με το LR - ο DT πιάνει τις non-linear αλληλεπιδράσεις (π.χ. «μικρά packets και υψηλό SYN count») με multi-split paths.

Feature importance. Η `Destination Port` κυριαρχεί συντριπτικά ($\sim 0,464$, δηλ. $\sim 46\%$), ακολουθούμενη από `Init_Win_bytes_backward` (0,121), `Bwd Packet Length Max` (0,077), `min_seg_size_forward` (0,067) και `Bwd Packets/s` (0,059). Τα top-3 features εξηγούν $\sim 66\%$ του δέντρου - η κυριαρχία λίγων features είναι χαρακτηριστική ενός μεμονωμένου δέντρου· το RF θα την εξομαλύνει.

Visualization του δέντρου. Το πραγματικό best DT έχει `max_depth = None` (unbounded), οπότε η πλήρης οπτικοποίηση είναι αδιάβαστη. Παρουσιάζουμε δύο εικόνες: (α) τα top-3 levels του actual best DT (Σχ. 4.2), και (β) ένα ξεχωριστό shallow tree `max_depth = 3` που εκπαιδεύσαμε μόνο για διδακτική απεικόνιση (Σχ. 4.3). Σε κάθε κόμβο εμφανίζονται: split rule (π.χ. `Destination Port ≤ 80,5`), entropy, αριθμός samples που κατέληξαν εκεί, κατανομή κλάσεων (`value=[...]`), και majority class (color-coded).



Σχήμα 4.2: Decision Tree (Σενάριο B, actual best) - top 3 levels. Το πραγματικό δέντρο μπορεί να φτάνει σε πολύ μεγαλύτερο βάθος, αλλά τα πρώτα splits χωρίζουν τις περισσότερες κλάσεις.



Σχήμα 4.3: Standalone shallow Decision Tree (max_depth = 3, entropy) εκπαιδευμένο ξεχωριστά για clean visualization. Όχι το best model - απλώς δείχνει πώς θα μοιάζει ένα ολόκληρο δέντρο 3 επιπέδων.

4.4 Random Forest

Διαίσθηση. Ensemble από πολλά δέντρα που διαφέρουν μεταξύ τους μέσω:

1. **Bagging:** κάθε δέντρο εκπαιδεύεται σε ένα bootstrap sample του training set (με replacement).
2. **Feature randomness:** σε κάθε split, εξετάζεται μόνο ένα τυχαίο υποσύνολο features (default $\sqrt{n}$).

Ένα μεμονωμένο δέντρο είναι high-variance· ο μέσος όρος πολλών στατιστικά ανεξάρτητων δέντρων μειώνει τη variance χωρίς να αυξάνει το bias. Επίσης, δεν υπερ-εμπιστεύεται κανένα μεμονωμένο

feature. Δεν απαιτείται scaling.

Grid. 18 combinations:

$$n_estimators \in \{50, 100, 200\}, \max_depth \in \{10, 20, \text{None}\}, \min_samples_split \in \{2, 10\}.$$

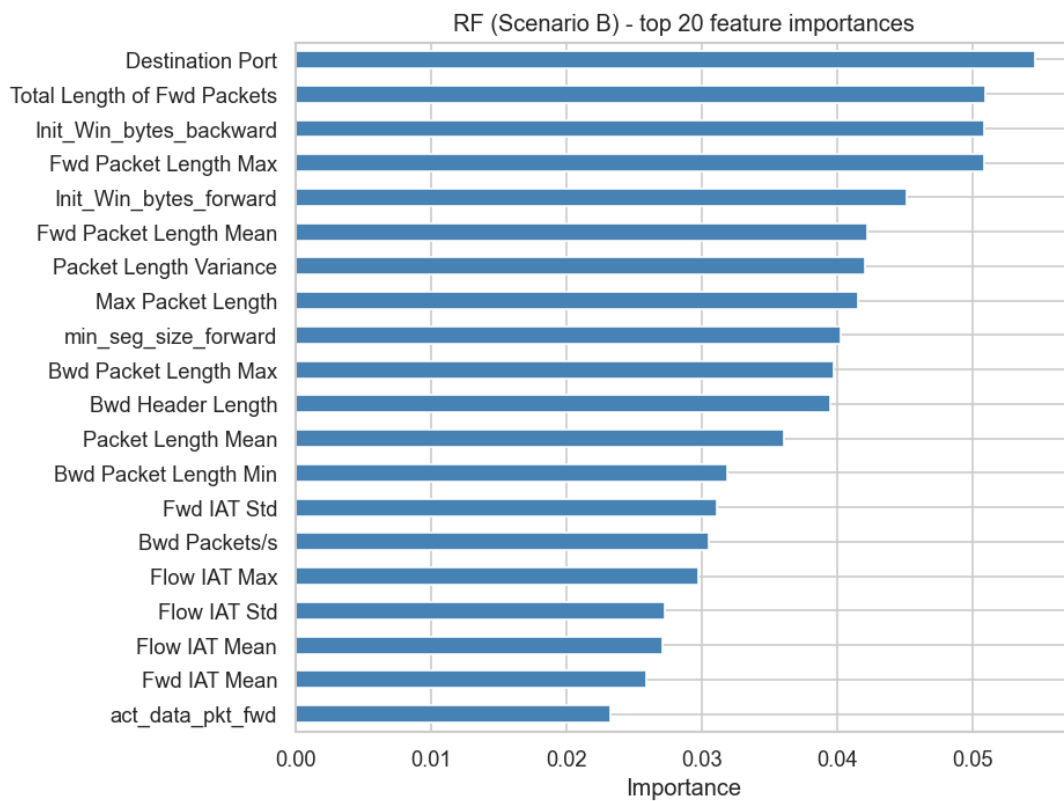
Αποτέλεσμα. Και τα δύο σενάρια επιλέγουν $n_estimators = 200$, $\max_depth = \text{None}$, $\min_samples_split = 2$.

Σενάριο	Accuracy	Precision _w	Recall _w	F1 _w	F1 _m
A (split)	0,9906	0,9953	0,9906	0,9927	0,7403
B (CV)	0,9906	0,9953	0,9906	0,9927	0,7403

Πίνακας 4.3: Random Forest - test metrics (500K balanced).

Τα δύο σενάρια δίνουν *ταυτόσημα* test metrics (ίδια best params). Το RF πετυχαίνει εξαιρετικό F1-weighted (0,9927) αλλά το F1-macro (0,7403) μένει κάτω από το DT (0,8705): το bagging + feature-randomness κάνει το RF πιο «συντηρητικό» στις rare κλάσεις σε σχέση με ένα βαθύ μεμονωμένο δέντρο. Σε production IDS που απαιτεί ισχυρή ανίχνευση rare attacks, θα εναλλάσσαμε το scoring σε F1-macro.

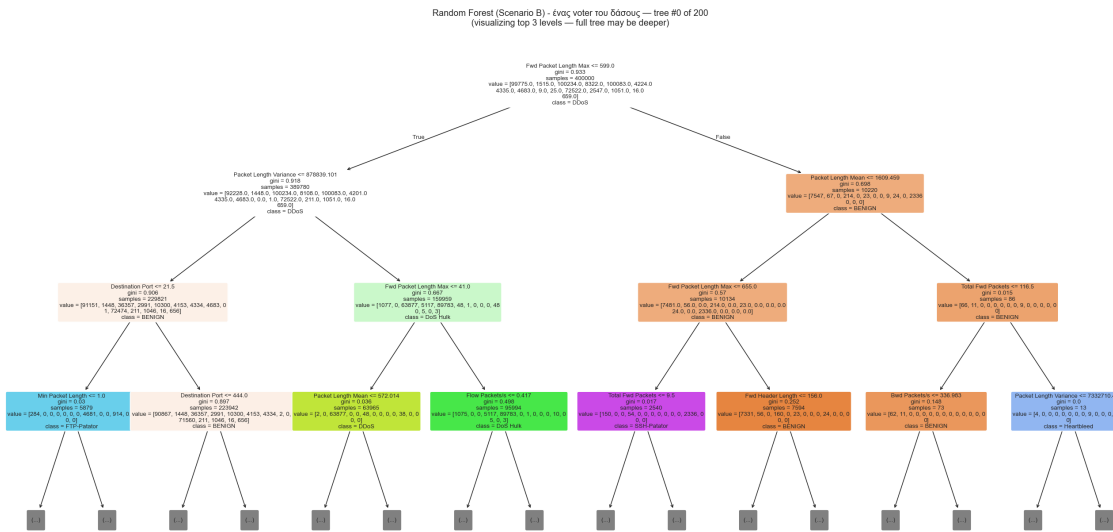
Feature importance. Σε αντίθεση με τον DT, το RF κατανέμει το importance: top-5 είναι **Destination Port** (0,055), **Total Length of Fwd Packets** (0,051), **Init_Win_bytes_backward** (0,051), **Fwd Packet Length Max** (0,051), **Init_Win_bytes_forward** (0,045). Δεν υπάρχει ένα feature-«μονάρχης»: το **Destination Port** έπεσε από ~ 46% (DT) σε ~ 5,5%, και τα top-10 κατανέμονται στη ζώνη ~ 4-5%. Αυτή η εξισορρόπηση είναι το signature του bagging/feature-randomness averaging.



Σχήμα 4.4: Top-20 feature importances του Random Forest (Σενάριο B).

Visualization ενός voter του δάσους. Το Σχ. 4.5 δείχνει τα top-3 levels ενός μεμονωμένου δέντρου από το best RF (δέντρο #0 από τα 100). Σημειώνουμε ότι:

- Το πρώτο split εδώ είναι **Bwd Packet Length Max** - διαφορετικό από το πρώτο split του standalone DT (**Destination Port**). Αυτό είναι ακριβώς το diversity που επιδιώκουμε με bagging+feature-randomness.
- Αν δείχναμε ένα δεύτερο δέντρο (π.χ. `tree_index=1`), η δομή θα ήταν ξανά διαφορετική.
- Η τελική πρόβλεψη του RF είναι *majority vote* (ή average των per-tree probabilities) πάνω σε όλα τα 100 δέντρα. Άρα ένα μεμονωμένο tree δεν αντιπροσωπεύει την «απόφαση» του RF.

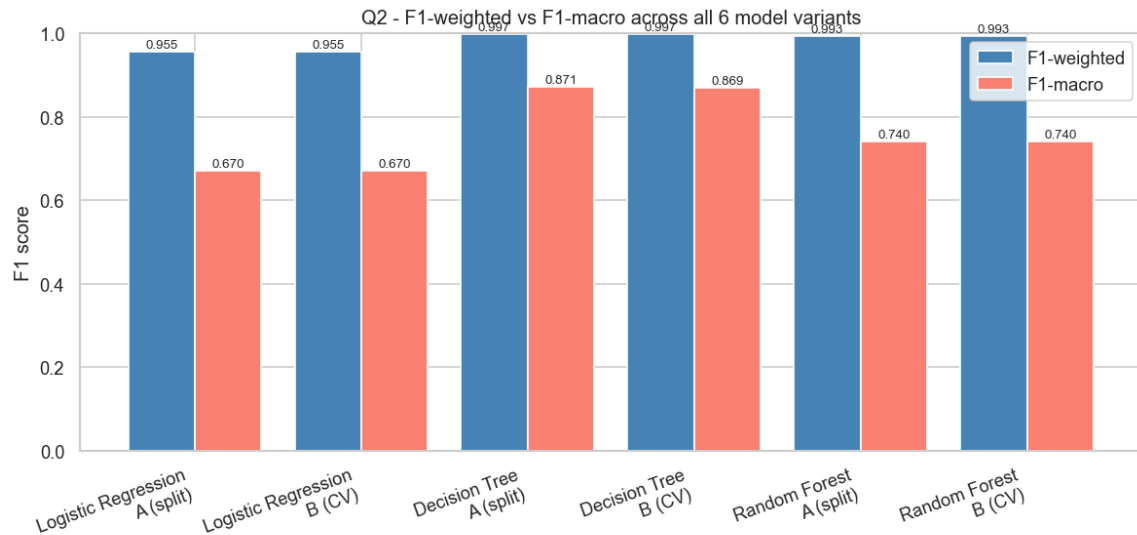


Σχήμα 4.5: Random Forest (Σενάριο B) - top 3 levels του πρώτου voter του δάσους (από τα 100 δέντρα συνολικά). Τα splits εδώ διαφέρουν από αυτά του standalone DT - σημείο της feature randomness που επιβάλλει το RF.

4.5 Συγκριτική Ανάλυση

Model	Σενάριο	Acc	Prec _w	Rec _w	F1 _w	F1 _m
Logistic Regression	A	0,9426	0,9761	0,9426	0,9548	0,6703
Logistic Regression	B	0,9426	0,9761	0,9426	0,9548	0,6703
Decision Tree	A	0,9973	0,9973	0,9973	0,9973	0,8705
Decision Tree	B	0,9971	0,9973	0,9971	0,9971	0,8691
Random Forest	A	0,9906	0,9953	0,9906	0,9927	0,7403
Random Forest	B	0,9906	0,9953	0,9906	0,9927	0,7403

Πίνακας 4.4: Q2 - σύγκριση 6 παραλλαγών στο ίδιο test set (100.000 rows, 500K balanced run).



Σχήμα 4.6: F1-weighted έναντι F1-macro και για τις 6 παραλλαγές. Η διαφορά $F1_w - F1_m$ είναι η «υπογραφή του imbalance»: πόσο «χρωστά» το μοντέλο την απόδοσή του στις μεγάλες κλάσεις.

4.5.1 Σύγκριση Balanced vs Proportional Sampling

Για να τεκμηριώσουμε την επιλογή του balanced sampling, τρέξαμε το ίδιο ακριβώς pipeline και με **proportional** sampling (ρεαλιστικό prior, $\sim 83\%$ BENIGN). Επειδή εκείνο το δείγμα λιμοκτονεί τις medium-rare κλάσεις και αντιστρέφει συχνότητες (βλ. 4.1), τα μοντέλα μαθαίνουν χειρότερα τις rare κλάσεις – ιδίως το γραμμικό LR:

Model	F1-macro (proportional)	F1-macro (balanced)
Logistic Regression	0,4764	0,6703
Random Forest	0,6668	0,7403
Decision Tree	0,8946	0,8705

Πίνακας 4.5: Επίδραση της στρατηγικής δειγματοληψίας στο F1-macro (και τα δύο runs στο ίδιο TARGET_ROWS=500_000). Το LR κερδίζει +0,19 και το RF +0,07 με balanced· το DT μένει πρακτικά ίδιο (μάλιστα οριακά υψηλότερο στο proportional, 0,8946 vs 0,8705 - ένα βαθύ δέντρο πιάνει τις rare κλάσεις ακόμα κι όταν είναι λίγες). Επιβεβαιώνει τον κανόνα: για supervised classification το training set πρέπει να αναπαριστά τα *σύνορα* των κλάσεων, όχι το πραγματικό prior.

Ο πλήρης πίνακας των 6 παραλλαγών για το **proportional** run (ίδιο pipeline, ίδιο TARGET_ROWS=500_000, μόνο BALANCED_SAMPLING=False) δίνεται παρακάτω για άμεση αντιπαραβολή με τον Πίνακα 4.4 (balanced):

Model	Σενάριο	Acc	Prec _w	Rec _w	F1 _w	F1 _m
Logistic Regression	A	0,8310	0,9598	0,8310	0,8819	0,4764
Logistic Regression	B	0,8310	0,9598	0,8310	0,8819	0,4764
Decision Tree	A	0,9978	0,9978	0,9978	0,9978	0,8946
Decision Tree	B	0,9978	0,9978	0,9978	0,9978	0,8946
Random Forest	A	0,9879	0,9941	0,9879	0,9904	0,6668
Random Forest	B	0,9879	0,9941	0,9879	0,9904	0,6668

Πίνακας 4.6: Q2 - σύγκριση 6 παραλλαγών στο proportional test set (100.000 rows, 500K proportional run· τα δύο σενάρια A/B έδωσαν ταυτόσημα best params και test metrics ανά μοντέλο).

Ανάγνωση. Στο proportional δείγμα ($\sim 83\%$ BENIGN) το LR καταρρέει στο macro (0,4764 έναντι 0,6703 balanced): το weighted F1 μένει υψηλό (0,8819) επειδή το μοντέλο «χρωστά» την απόδοσή του στη μεγάλη κλάση BENIGN, ενώ οι medium-rare κλάσεις (Bot, Brute Force) που λιμοκτονούν στο proportional δείγμα μένουν ουσιαστικά αδιάγνωστες. Το RF ακολουθεί το ίδιο μοτίβο σε ηπιότερο βαθμό (0,6668 $\rightarrow$ 0,7403). Το DT, αντίθετα, παραμένει ανθεκτικό (0,8946) - ένα μη γραμμικό βαθύ δέντρο απομονώνει τις rare υπογραφές ακόμη και με λίγα παραδείγματα, γι' αυτό η σειρά νικητών (DT > RF > LR) δεν αλλάζει.

4.5.2 Απαντήσεις στις Τέσσερις Ερωτήσεις της Εκφώνησης

1. Ποιος αλγόριθμος αποδίδει καλύτερα; Νικητής είναι το **Decision Tree**: F1-weighted 0,9973 (A) / 0,9971 (B) και F1-macro **0,8705** / 0,8691 - το υψηλότερο και στις δύο μετρικές. Ακολουθεί το **Random Forest** (F1-weighted 0,9927, F1-macro 0,7403) και τελευταίο το **Logistic Regression** (F1-weighted 0,9548, F1-macro 0,6703). Το ότι το DT ξεπερνά το RF στο macro οφείλεται στο ότι ένα βαθύ δέντρο κόβει καθαρά τις μικρές κλάσεις, ενώ το RF (bagging + feature-randomness) είναι πιο «συντηρητικό». Το LR, ως γραμμικό μοντέλο, δεν διαχωρίζει non-linearly separable κλάσεις - αλλά με balanced sampling το F1-macro του ανέβηκε σημαντικά (από 0,48 σε 0,67, βλ. 4.5.1).

2. Δίνει το Grid Search με CV διαφορετικά hyperparameters σε σχέση με split-based; **LR:** ίδια best params και στα δύο σενάρια ($C = 10$, lbfgs, max_iter= 500) - ταυτόσημα test metrics. **DT:** ελαφρώς διαφορετικά (A: max_depth=None, B: max_depth= 20, κοινό entropy/min_samples_split= 2) με πρακτικά ισοδύναμο test F1 (flat plateau). **RF:** ίδια best params (n_estimators= 200, max_depth=None, min_samples_split= 2). Με μεγάλο validation set (100K rows) η split-based επιλογή συμφωνεί ουσιαστικά με τον CV μέσο. Η CV παραμένει πιο αξιόπιστη ως ranking (std $\sim$ 0,0002-0,004) αλλά πολύ πιο αργή (RF-B: 2174s vs RF-A: 362s· LR-B: 1718s vs LR-A: 454s). **Γενικό μάθημα:** όταν το dataset είναι περιορισμένο ή οι κλάσεις σπάνιες, η CV είναι πιο αξιόπιστη.

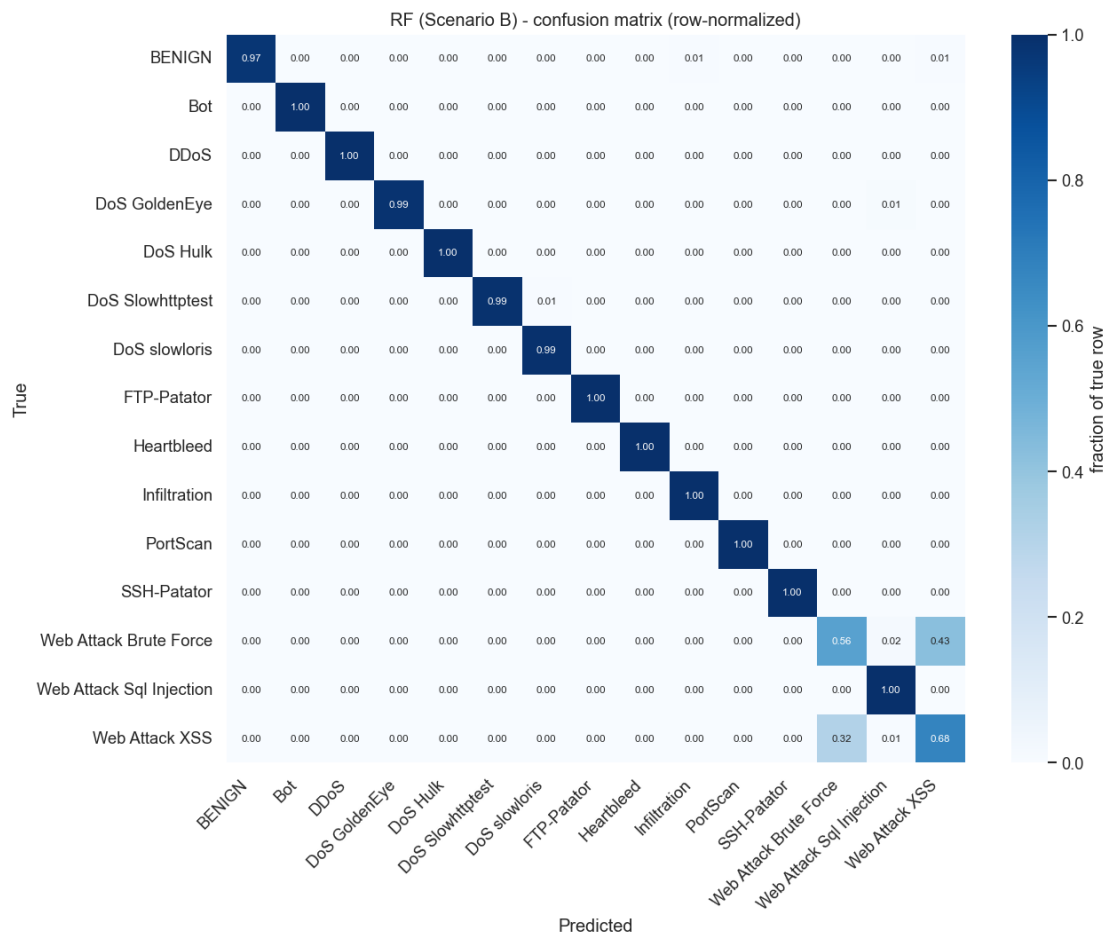
3. Ποιες attack types είναι οι πιο δύσκολες; Από τις row-normalized confusion matrices (Figs. 4.7, 4.8):

- **Web Attacks (Brute Force / XSS / SQL Injection):** συγχέονται μεταξύ τους και με BENIGN. Αναπαράγουν normal HTTP traffic με μικρές υπογραφές.
- **Infiltration (36 rows) και Heartbleed (11 rows):** λόγω περιορισμένων training data τα signatures είναι αναξιόπιστα. Συχνά κατηγοριοποιούνται ως άλλη high-frequency DoS κλάση.
- **Bot:** μπερδεύεται με BENIGN επειδή το command-and-control traffic μπορεί να μιμείται κανονικές HTTP/HTTPS calls.

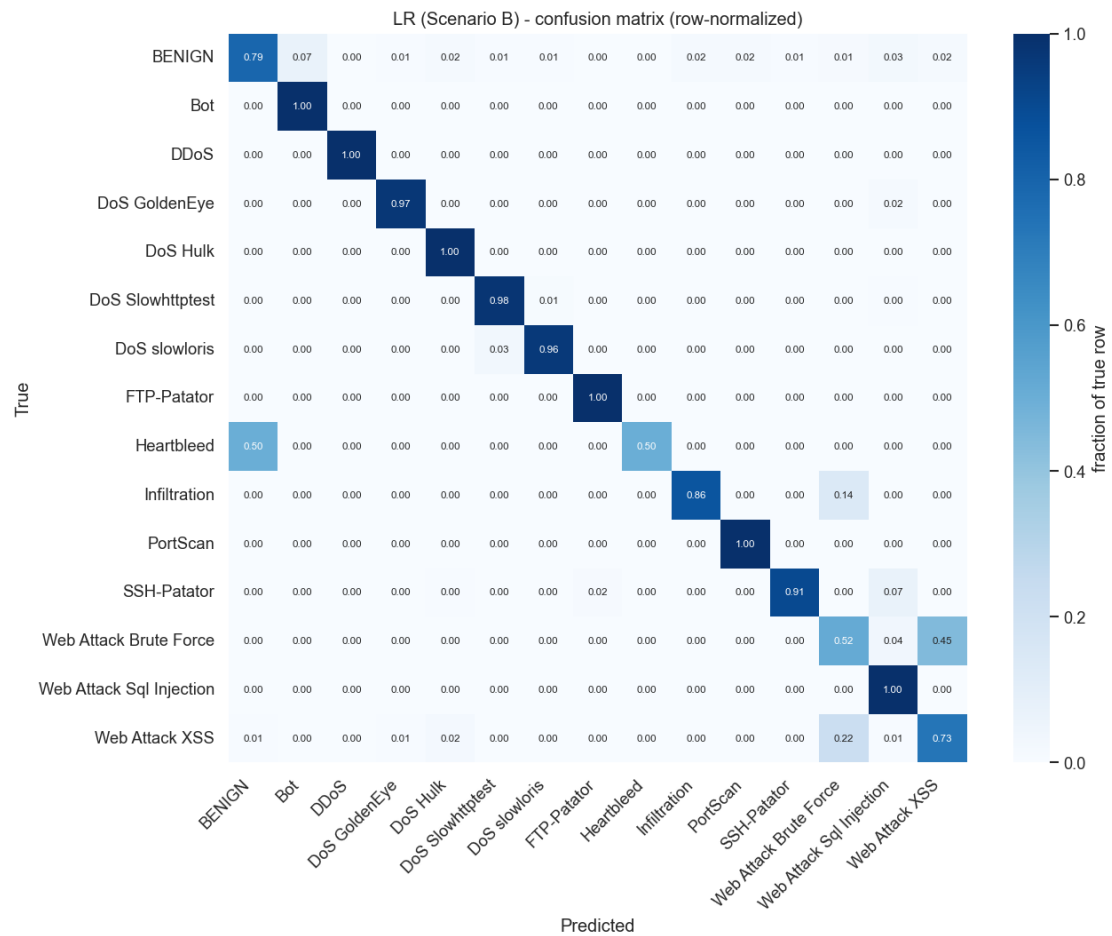
Οι «εύκολες» κλάσεις είναι DDoS, DoS Hulk, PortScan, FTP-Patator, SSH-Patator - έχουν πολύ διακριτές υπογραφές (rate spikes, αποκλίνοντα packet sizes, καταιγίδες SYN χωρίς ACK).

4. Πώς επηρεάζει το class imbalance τα αποτελέσματα; Σε όλα τα μοντέλα ισχύει $F1_w > F1_m$, και το χάσμα δείχνει την ποιότητα στις rare κλάσεις: **DT** 0,9973 vs 0,8705 (gap $\sim 0,13$), **RF** 0,9927 vs 0,7403 (gap $\sim 0,25$), **LR** 0,9548 vs 0,6703 (gap $\sim 0,28$). Το weighted τιμάει τις τεράστιες κλάσεις (BENIGN/DDoS/DoS Hulk) που όλοι πιάνουν εύκολα· το macro «τιμωρεί» τις rare. Σημαντικό: η ίδια η στρατηγική δειγματοληψίας επηρεάζει δραστικά αυτό το χάσμα – το balanced sampling ανέβασε το LR macro κατά +0,19 (Πίν. 4.5). Η επιλογή μετρικής εξαρτάται από την εφαρμογή:

- IDS που θέλει να πιάνει *κάθε* attack type $\rightarrow$ F1-macro.
- Production filter που σταθμίζεται με τη συχνότητα traffic $\rightarrow$ F1-weighted.



Σχήμα 4.7: Random Forest (Σενάριο B) - row-normalized confusion matrix. Οι διαγώνιοι κοντά στο 1 δείχνουν per-class recall· οι off-diagonal entries δείχνουν confusion patterns.



Σχήμα 4.8: Logistic Regression (Σενάριο B) - row-normalized confusion matrix. Σημαντικά χειρότερη ποιότητα στις Web-Attack και Bot κλάσεις σε σχέση με το RF.

4.6 Σύνοψη Q2

Μέγεθος	Τιμή
Sample (balanced, TARGET_ROWS=500K)	500.000 rows
Train / Val / Test	300.000 / 100.000 / 100.000
Αλγόριθμοι	LR, DT, RF
Σενάρια	A (split), B (CV)
LR grid combinations	8
DT grid combinations	24
RF grid combinations	18
Best model (F1-weighted)	DT A: 0,9973
Best model (F1-macro)	DT A: 0,8705
Worst F1-macro	LR: 0,6703

Πίνακας 4.7: Βασικά αποτελέσματα του Q2 (500K balanced run).

Κεφάλαιο 5

Ερώτημα 3: Συσταδοποίηση

Στο κεφάλαιο αυτό αποκρύπτουμε τη στήλη `Label` και εφαρμόζουμε τρεις unsupervised αλγορίθμους clustering (K-Means, Agglomerative Hierarchical, DBSCAN) στο PCA-μειωμένο feature space. Στόχος: να δούμε αν η ενδογενής δομή των δεδομένων αντιστοιχεί στις γνωστές attack categories - χωρίς supervisory signal. Αξιολογούμε με Silhouette και Davies-Bouldin Index (όπως ζητά η εκφώνηση), και επιβεβαιώνουμε την αντιστοίχιση clusters $\leftrightarrow$ labels post-hoc μέσω heatmap.

5.1 Preprocessing και PCA

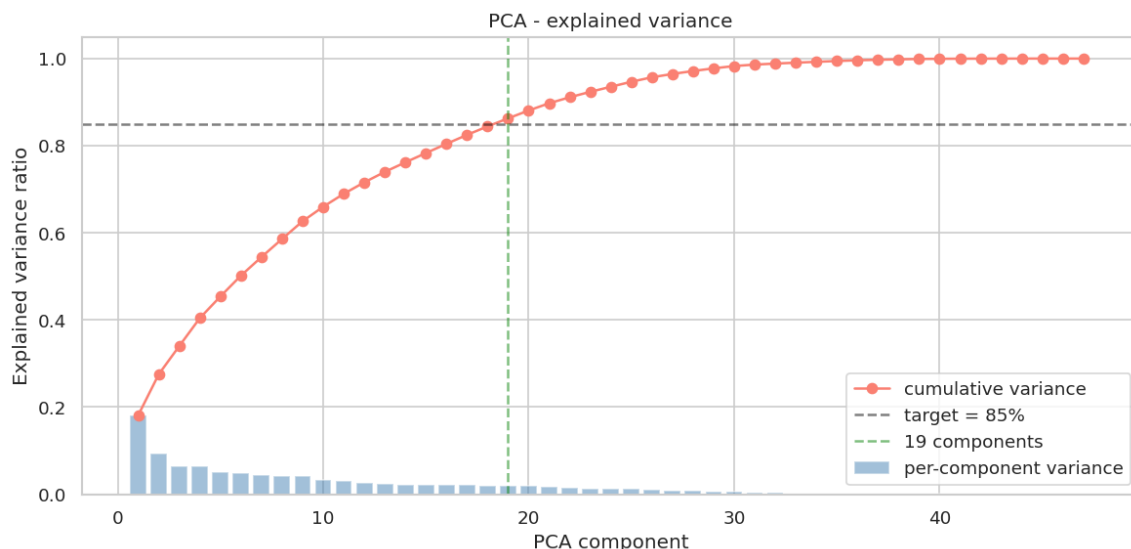
Χρησιμοποιούμε την ίδια `subsample_to_target` με το Q2, αλλά με **αντίθετη** επιλογή: `BALANCED_SAMPLING=False` (proportional), `TARGET_ROWS=500.000`. Γιατί το αντίθετο από το Q2: το clustering ψάχνει τη **φυσική δομή** των δεδομένων, οπότε θέλουμε να δει την πραγματική πυκνότητα ($\sim 83\%$ BENIGN). Ένας supervised classifier (Q2) χρειάζεται balanced για να μάθει τα σύνορα όλων των κλάσεων· ένας unsupervised αλγόριθμος (Q3) θα ήταν παραπλανητικό να τρέξει σε τεχνητά εξισορροπημένα δεδομένα, γιατί θα «μηχανευόμασταν» τη δομή που ελπίζουμε να βρούμε. Οι rare classes (Heartbleed, SQL Injection, Infiltration) διατηρούνται πλήρως μέσω του rare-floor. Η σύγκριση με balanced sampling δίνεται στο 5.5.

Γιατί StandardScaler είναι κρίσιμο εδώ; Όλοι οι αλγόριθμοι clustering που χρησιμοποιούμε είναι distance-based: η ευκλείδεια απόσταση μεταξύ points ορίζει τα clusters. Χωρίς scaling, ένα feature σε scale 10^6 (Bytes/s) θα κυριαρχούσε 100% τη μέτρηση - ουσιαστικά το clustering θα ήταν ταξινόμηση μόνο πάνω σε αυτό το ένα feature. Το StandardScaler εξασφαλίζει ότι κάθε feature συνεισφέρει εξίσου.

PCA - dimensionality reduction. Δύο λόγοι:

- *Curse of dimensionality*: σε υψηλές διαστάσεις (47 features), όλες οι αποστάσεις τείνουν να γίνονται παρόμοιες, και η έννοια του «κοντινός γείτονας» χάνει νόημα. Το PCA συμπυκνώνει την variance σε λιγότερα orthogonal axes, αποκαθιστώντας το distance signal.
- *Ταχύτητα*: K-Means και DBSCAN κλιμακώνονται γραμμικά ως προς τη διάσταση. Από 47 $\rightarrow$ 19 components είναι $\sim 2,5\times$ speedup.

Σημείωση επιτάχυνσης. Το Q3 τρέχει PCA, K-Means και DBSCAN στη GPU μέσω RAPIDS cuML (με fallback σε scikit-learn). Λεπτομέρειες και μετρήσεις στο Κεφάλαιο B'.

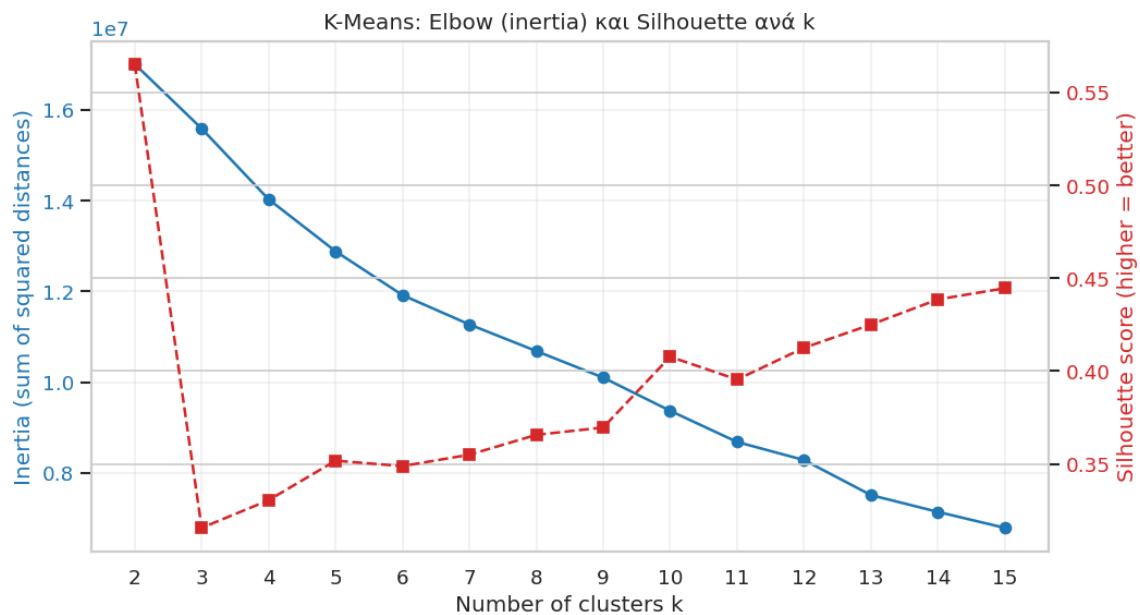


Σχήμα 5.1: PCA explained variance: per-component (μπλε bars) και cumulative (κόκκινη γραμμή). Με **19 components** καλύπτουμε $\sim 86,3\%$ της variance· επιλέγουμε αυτό ως input για όλους τους αλγορίθμους clustering. Παρατηρήστε ότι ο πρώτος component συλλαμβάνει μόνο $\sim 18\%$: δεν υπάρχει ένα κυρίαρχο axis. Η information είναι κατανεμημένη σε πολλά components.

5.2 K-Means

Διαίσθηση: Επιλογή k . Επανάληψη μέχρι σύγκλιση: (1) *Ανάθεση* κάθε σημείου στο πιο κοντινό κεντροειδές, (2) *Ενημέρωση* κάθε κεντροειδούς ως μέσο όρο των σημείων του. Συγκλίνει σε τοπικό ελάχιστο του inertia (sum of squared distances). Υποθέτει **σφαιρικά clusters παρόμοιου μεγέθους**.

Επιλογή k - Elbow + Silhouette. Σαρώνουμε $k \in [2, 15]$. Το inertia (αριστερός άξονας) μειώνεται μονοτονικά από $\sim 17,0M$ ($k = 2$) σε $\sim 6,8M$ ($k = 15$) - χωρίς σαφή αγκώνα. Το silhouette (δεξιός άξονας) είναι **υψηλότερο στο $k = 2$** (0,5653), πέφτει απότομα στο $k = 3$ (0,3156) και ανεβαίνει σταδιακά ξανά ως 0,4446 στο $k = 15$ (δευτερεύον τοπικό μέγιστο της καμπύλης· η αυτοτελής επαναποτίμηση στο $k = 15$ δίνει 0,4300, Πίν. 5.1) - αλλά **το global maximum παραμένει στο $k = 2$** .



Σχήμα 5.2: K-Means - inertia (μπλε) και silhouette (κόκκινο) ανά k . Το silhouette κορυφώνεται στο $k = 2$ (normal vs volumetric attacks)· το $k = 15$ (= #labels) τρέχει ως reference.

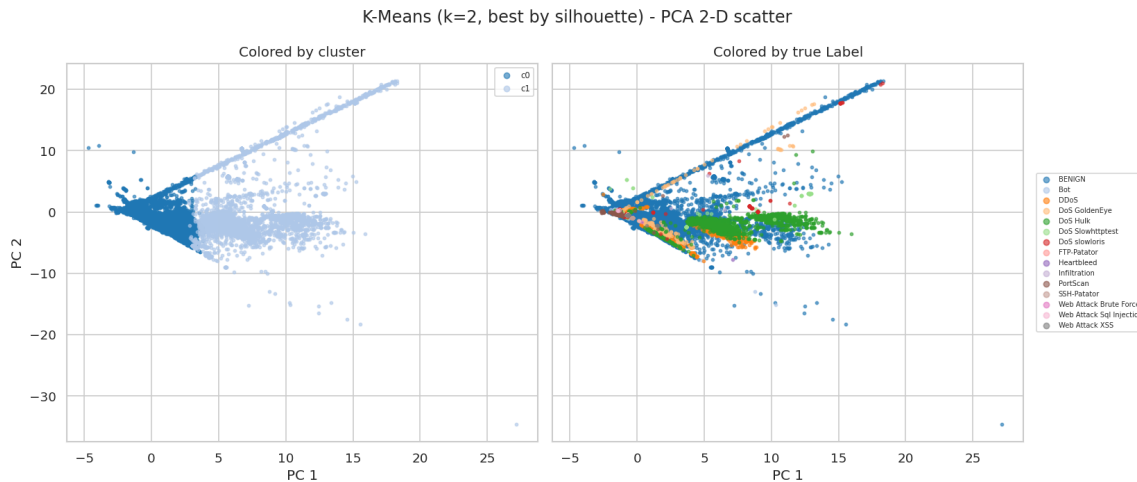
Best $k = 2$ και τι σημαίνει. Σε proportional sampling το silhouette-best k είναι 2, όχι 15. Από το cluster-to-Label heatmap, το split είναι ερμηνεύσιμο: το cluster **C0** είναι ~ 89% BENIGN («κανονική κίνηση»), ενώ το **C1** κυριαρχείται από DoS Hulk (~ 49%) και DDoS (~ 12%) («ογκώδεις / volumetric επιθέσεις»). Δηλαδή η ισχυρότερη γεωμετρική δομή είναι μια δυαδική διχοτόμηση *normal vs volumetric DoS*.

Σημαντική διευκρίνιση – interpretation του $k = 2$

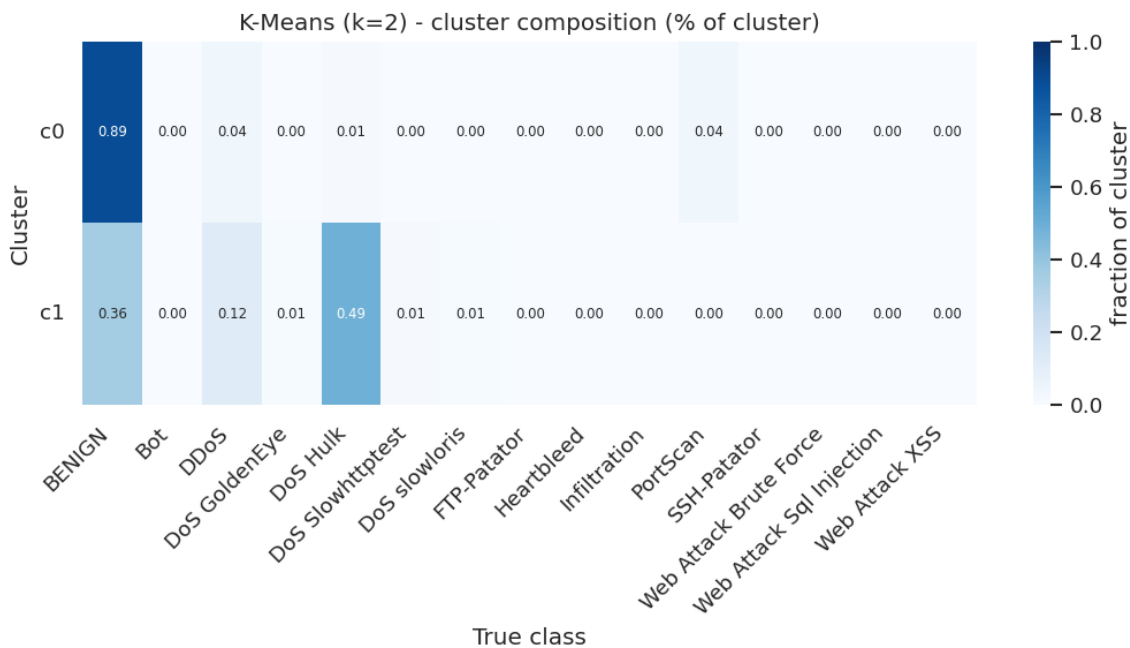
Το ότι κερδίζει το $k = 2$ **δεν** σημαίνει ότι υπάρχουν μόνο 2 τύποι επιθέσεων. Σημαίνει ότι οι 15 κλάσεις **δεν είναι καθαρά διαχωρίσιμες ως 15 γεωμετρικές «μπάλες»** σε αυτόν τον PCA χώρο, υπό proportional (83% BENIGN) δειγματοληψία. Το ισχυρότερο γεωμετρικό σήμα είναι δυαδικό (κανονική κίνηση vs flood)· η λεπτότερη 15-way δομή *υπάρχει αλλά είναι μπερδεμένη / επικαλυπτόμενη* – κάτι που είναι από μόνο του ένα ειλικρινές και υπερασπίσιμο εύρημα. (Επιβεβαιώνεται από extended αναζήτηση k έως 50: το silhouette πλησιάζει το $k = 2$ μόνο όταν το K-Means σπάει το σύνολο σε δεκάδες μικροσκοπικά clusters outliers – τεχνητή «βελτίωση», όχι πραγματική δομή.)

Configuration	n_clusters	Silhouette ↑	Davies-Bouldin ↓
K-Means $k = 2$ (best by silhouette)	2	0,5653	1,4935
K-Means $k = 15$ (reference = #labels)	15	0,4300	0,8026

Πίνακας 5.1: K-Means - test metrics (proportional 500K). Best $k = 2$ έχει το υψηλότερο silhouette· το $k = 15$ reference έχει καλύτερο (χαμηλότερο) Davies-Bouldin – οι δύο μετρικές διαφωνούν σε ασύμμετρα δεδομένα.



Σχήμα 5.3: K-Means $k = 2$ (best) - PCA 2-D scatter (rendering sample 50K points). Αριστερά: colored by cluster. Δεξιά: colored by true Label. Με 83% BENIGN, ο χώρος κυριαρχείται από το «κανονικό» cloud· η διχοτόμηση normal-vs-volumetric είναι ο πιο ευδιάκριτος διαχωρισμός.



Σχήμα 5.4: K-Means $k = 2$ - cluster composition heatmap. Row-normalized. **c0**: $\sim 89\%$ BENIGN (normal traffic). **c1**: $\sim 49\%$ DoS Hulk + $\sim 12\%$ DDoS (volumetric attacks). Ένας ερμηνεύσιμος, αν και χονδρικός, διαχωρισμός.

5.3 Hierarchical (Agglomerative)

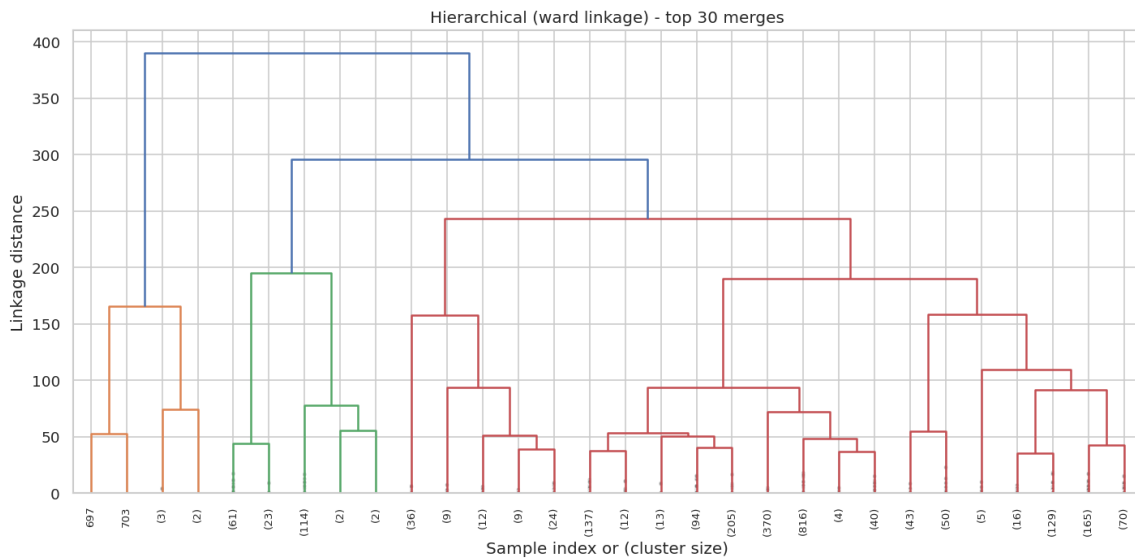
Διαίσθηση. Bottom-up: ξεκινάμε με κάθε point ως ένα cluster· σε κάθε βήμα ενώνουμε τα δύο πιο κοντινά. Παράγει ένα *dendrogram*. Κόβουμε το dendrogram σε κατάλληλο height για να πάρουμε k clusters.

Linkage methods ορίζουν την «απόσταση μεταξύ clusters»:

- **single**: minimum point-to-point distance (chaining effect).

- **complete**: maximum point-to-point distance (συμπαγή σφαιρικά clusters).
- **average**: μέσος όρος όλων των ζευγών (συμβιβασμός).
- **ward**: minimum-variance criterion - ενώνει clusters που ελαχιστοποιούν την αύξηση της within-cluster variance.

Computational constraint. Η AgglomerativeClustering απαιτεί ολόκληρο τον $O(n^2)$ πίνακα αποστάσεων: για 500K rows αυτό είναι απαγορευτικό σε μνήμη (~ 2 TB) - και δεν υπάρχει cuML/GPU υλοποίηση. Παίρνουμε stratified subsample 2.468 **rows** (~ 70 MB distance matrix) - αρκετό για διαβάσιμο dendrogram και σταθερό linkage comparison. (Το hierarchical είναι ο μόνος αλγόριθμος που δεν κλιμακώνεται σε κανένα hardware.)



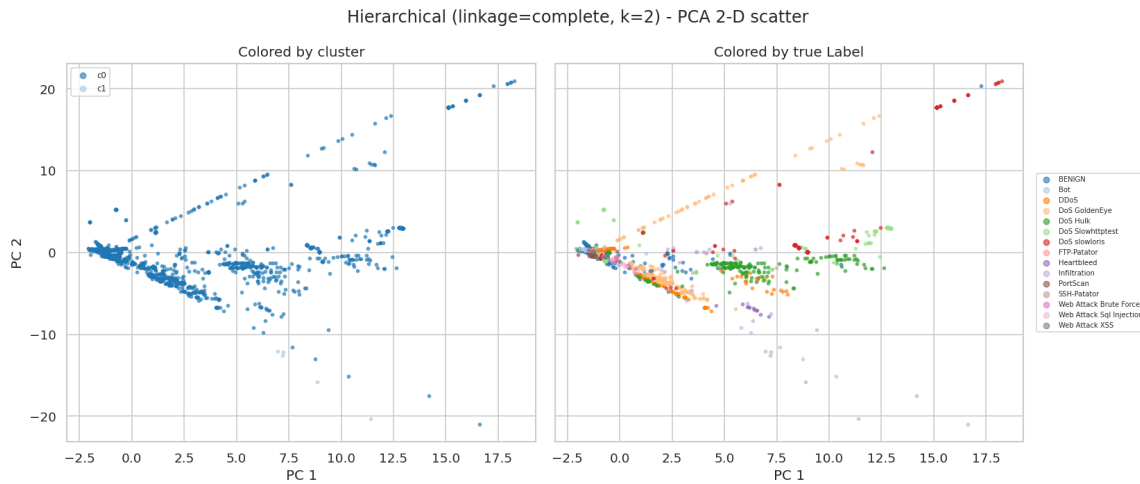
Σχήμα 5.5: Dendrogram (ward linkage, top 30 merges). Φαίνεται καθαρά διπλή ιεραρχική δομή: ένα μεγάλο σύνολο merges σε χαμηλό height (πορτοκαλί - κανονικό traffic και κοντινά attacks) και ένα απομονωμένο cluster (πράσινο - probably attack patterns που διαφέρουν δραματικά). Η μεγάλη κάθετη γραμμή στο height ~ 800 δείχνει την τελική συγχώνευση των δύο sub-trees.

Linkage comparison με $k = 2$ (= silhouette-best K-Means k , για consistency):

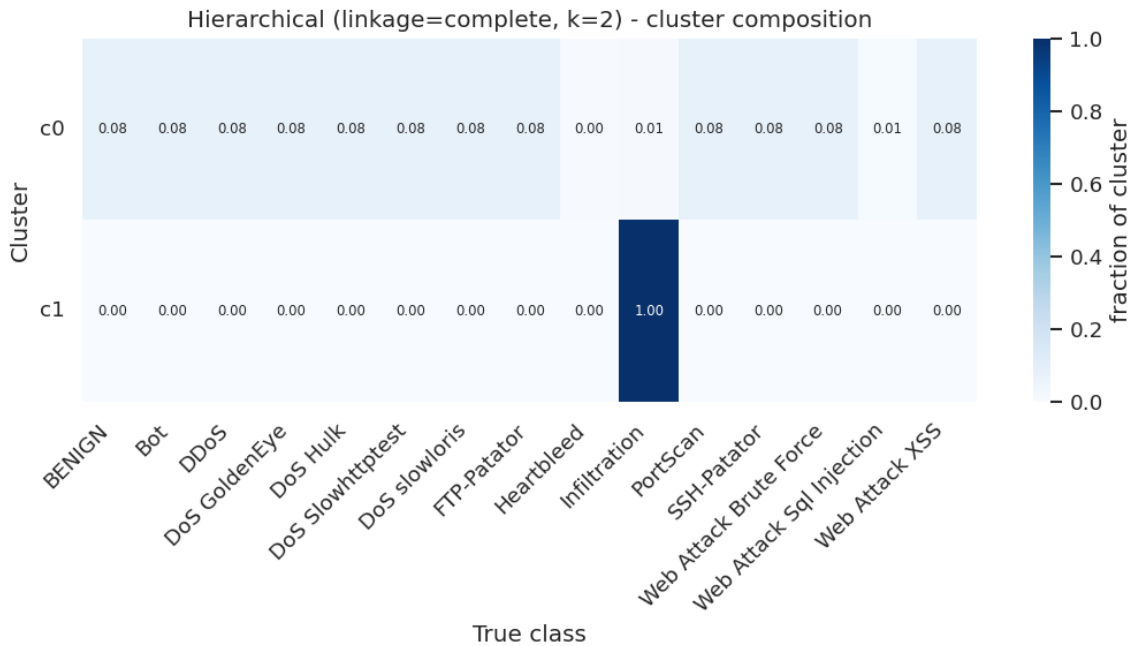
Linkage	Silhouette $\uparrow$	Davies-Bouldin $\downarrow$
ward	0,9168	0,5021
complete	0,9201	0,2488
average	0,9201	0,2488

Πίνακας 5.2: Hierarchical Clustering - σύγκριση linkage methods ($k = 2$, 2.468-row stratified subsample). Best: complete/average (ταυτόσημα).

Προσοχή στην ερμηνεία. Τα complete/average δίνουν πολύ υψηλό silhouette ($\sim 0,92$) - αλλά σε $k = 2$ πάνω σε δείγμα με $\sim 83\%$ BENIGN, αυτό αντανακλά έναν εύκολο γεωμετρικό διαχωρισμό (ένα μικρό πυκνό group απομονώνεται από τον υπόλοιπο όγκο), όχι αντιστοίχιση με τις 15 κλάσεις. Όπως και αλλού, ένα υψηλό silhouette score πρέπει να διασταυρώνεται με το τι περιέχουν πραγματικά τα clusters (βλ. heatmap) - δεν είναι από μόνο του απόδειξη ουσιαστικής δομής.



Σχήμα 5.6: Hierarchical (best linkage, $k = 2$) - PCA 2-D scatter στο 2.468-row subsample.



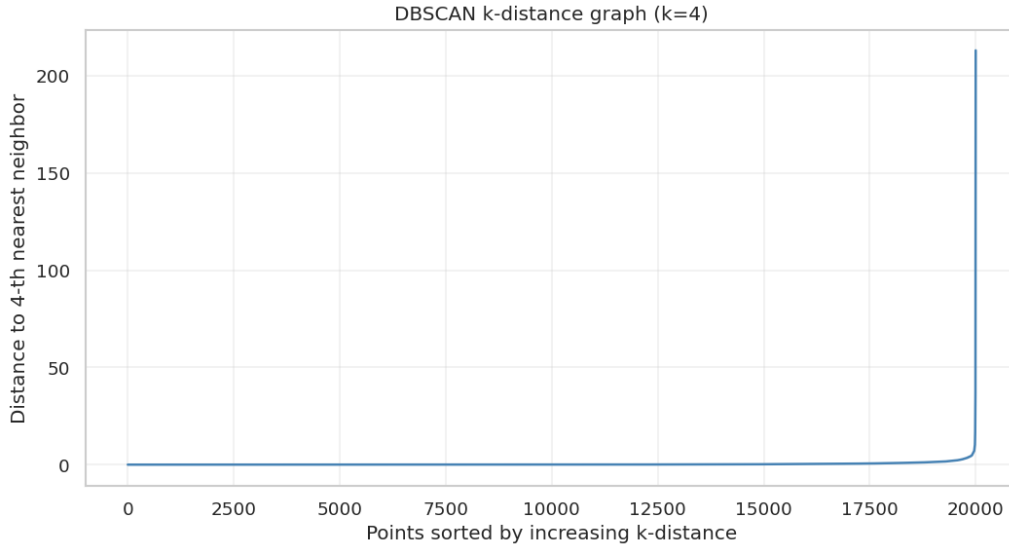
Σχήμα 5.7: Hierarchical (best linkage, $k = 2$) - cluster composition heatmap.

5.4 DBSCAN

Διαίσθηση. Density-based clustering. Δεν χρειάζεται k . Έννοιες: *core point* ($\geq \text{min_samples}$ γείτονες μέσα σε ε -ακτίνα), *border point*, *noise* (label = -1). Δύο core points εντός ε ανήκουν στο ίδιο cluster.

Πλεονεκτήματα. Ανακαλύπτει clusters αυθαίρετου σχήματος (όχι μόνο σφαιρικά)· ανθεκτικό σε outliers (τα ταξινομεί ως noise). **Μειονεκτήματα.** Ευαίσθητο σε ε και min_samples · δυσκολεύεται με clusters διαφορετικής πυκνότητας (μία ε δεν ταιριάζει σε όλα).

Επιλογή ε - k-distance graph. Για κάθε point, απόσταση προς τον $k = 4$ πιο κοντινό γείτονα, sorted ascending. Ο «αγκώνας» της καμπύλης δείχνει ένα καλό ε - κάτω από τον αγκώνα είναι dense regions.



Σχήμα 5.8: k-distance graph ($k = 4$) πάνω σε στρωμένο sample 20.000 rows. Η κύρια καμπύλη παραμένει σχεδόν επίπεδη μέχρι το $\sim 80\%$ percentile και έπειτα εκτοξεύεται απότομα: σαφής αγκώνας στο $\varepsilon \approx 0,3-0,5$, κάτι που επιβεβαιώνει το sweep μας.

Sweep $\varepsilon \times \text{min_samples}$. $5 \times 3 = 15$ combinations σύμφωνα με την εκφώνηση:

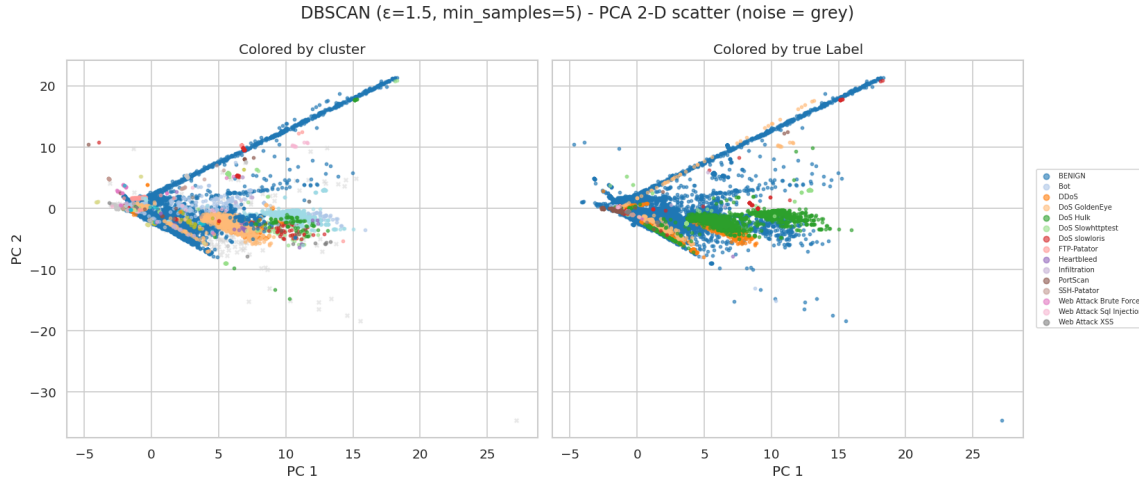
ε	min_samples	n_clusters	n_noise	Sil. $\uparrow$	DB $\downarrow$
0,3	5	1.526	30.943	0,0716	0,5882
0,3	10	776	40.542	0,0416	0,5836
0,3	20	479	50.579	0,0988	0,5815
0,5	5	1.053	17.470	-0,0049	0,6576
0,5	10	606	24.163	0,0273	0,6625
0,5	20	347	32.367	0,0707	0,6599
0,8	5	731	8.998	-0,0166	0,7210
0,8	10	400	13.241	0,0052	0,7127
0,8	20	248	18.655	0,0250	0,7144
1,0	5	432	5.957	-0,3374	0,5991
1,0	10	253	8.977	-0,3027	0,5855
1,0	20	154	12.937	-0,1299	0,5267
1,5	5	225	2.649	0,1051	0,5493
1,5	10	129	3.998	0,0107	0,5547
1,5	20	93	5.914	0,0823	0,4855

Πίνακας 5.3: DBSCAN sweep (proportional 500K) - όλες οι 15 παραλλαγές. Best silhouette: $\varepsilon = 1,5, \text{min_samples} = 5$ (μόλις 0,105). Best Davies-Bouldin: $\varepsilon = 1,5, \text{min_samples} = 20$. Όλα τα silhouette είναι χαμηλά (αρκετά αρνητικά στα μεγάλα ε): το DBSCAN δυσκολεύεται σε αυτό το πυκνό, ανομοιόμορφο dataset.

Παρατηρήσεις.

- **Πάρα πολλά clusters** σε όλες τις configurations (93-1.526), πολύ περισσότερα από τα 15 πραγματικά labels. Αυτό σημαίνει ότι η ενδογενής δομή έχει πολλές sub-populations - διαφορετικά *flavors* του ίδιου attack type ή του BENIGN.

- **Trade-off noise vs cluster quality:** μικρό $\varepsilon \Rightarrow$ τεράστιο noise (30K-51K rows με $\varepsilon = 0,3$). Μεγάλο $\varepsilon \Rightarrow$ λίγο noise αλλά αρνητικό silhouette (έως $-0,34$ στο $\varepsilon = 1,0$) - points σε «λάθος» cluster.
- Επιλέγουμε ως best το $\varepsilon = 1,5$, $\text{min_samples} = 5$ (highest silhouette του sweep, μόλις $0,105$). Ακόμα κι αυτό είναι πολύ κάτω από το K-Means $k = 2$ ($0,565$): καθαρό μήνυμα ότι το density-based clustering δυσκολεύεται εδώ.



Σχήμα 5.9: DBSCAN ($\varepsilon = 1,5$, $\text{min_samples} = 5$) - PCA 2-D scatter. Τα grey x-markers είναι *noise points* (2.649 rows, $\sim 0,5\%$ σε αυτό το config· στα μικρά ε το noise εκτοξεύεται σε 30K-51K).

5.5 Συγκριτική Ανάλυση

Algorithm	Config	n_clust	n_noise	Sil. $\uparrow$	DB $\downarrow$
K-Means	$k = 2$ (best)	2	0	0,5653	1,4935
K-Means	$k = 15$ (reference)	15	0	0,4300	0,8026
Hierarchical	ward, $k = 2$	2	0	0,9168	0,5021
Hierarchical	complete, $k = 2$	2	0	0,9201	0,2488
Hierarchical	average, $k = 2$	2	0	0,9201	0,2488
DBSCAN	$\varepsilon = 1,5$, $ms = 5$	225	2.649	0,1051	0,5493
DBSCAN	$\varepsilon = 0,3$, $ms = 20$	479	50.579	0,0988	0,5815

Πίνακας 5.4: Q3 - σύγκριση best configurations ανά αλγόριθμο (proportional 500K). *Caveats:* το Hierarchical εκπαιδεύτηκε σε 2.468-row subsample λόγω $O(n^2)$ memory, και το υψηλό του silhouette σε $k = 2$ αντανακλά εύκολο διαχωρισμό, όχι αντιστοίχιση με labels.

5.5.1 Απαντήσεις στις Ερωτήσεις της Εκφώνησης

1. Αντιστοιχούν τα clusters σε meaningful attack categories; Μερικώς - και η απάντηση εξαρτάται από την κλίμακα. Από τα cluster-to-Label heatmaps:

- **K-Means (best $k = 2$):** ο διαχωρισμός είναι ερμηνεύσιμος αλλά χονδρικός - $c0 \sim 89\%$ BENIGN, $c1$ κυριαρχείται από DoS Hulk/DDoS. Η ισχυρότερη δομή είναι normal-vs-volumetric-DoS, όχι 15 ξεχωριστές κατηγορίες.

- **DBSCAN** κατακερματίζει σε εκατοντάδες clusters - πολλά σχεδόν καθαρά (100% BENIGN pockets, ή σχεδόν καθαρά DoS Hulk), που δείχνει ότι η **BENIGN** δεν είναι ομοιογενής αλλά μείγμα πολλών προφίλ (web/email/bulk).
- Οι rare attacks (Heartbleed, SQL Injection, Infiltration) σπάνια σχηματίζουν δικό τους σταθερό cluster: είναι πολύ λίγες για να αναδειχθούν γεωμετρικά μέσα στο proportional δείγμα.

2. Ποιος αλγόριθμος συλλαμβάνει καλύτερα την true structure; Με βάση μόνο το silhouette, το **Hierarchical** (complete/average, 0,920) φαίνεται καλύτερο - **αλλά είναι παραπλανητικό**: σε $k = 2$ το ψηλό score προκύπτει από έναν εύκολο γεωμετρικό διαχωρισμό σε μικρό subsample, όχι από αντιστοίχιση με κλάσεις. **K-Means** $k = 2$ (silhouette 0,565, 0 noise) δίνει τον πιο έντομο χονδρικό διαχωρισμό: το **DBSCAN best** (silhouette μόλις 0,105) είναι σαφώς το πιο αδύναμο - το density-based clustering δυσκολεύεται στο πυκνό dataset. **Συμπέρασμα**: κανένας αλγόριθμος δεν ανακτά καθαρά τις 15 κατηγορίες σε αυτό το ρεαλιστικό (83% BENIGN) δείγμα: η κυρίαρχη δομή είναι «normal vs volumetric DoS». Το silhouette πρέπει να διαβάζεται μαζί με το περιεχόμενο των clusters, αλλιώς ένα degenerate outlier-split «κερδίζει» λάθος.

3. Τι ανακαλύπτουμε χωρίς labels που δεν φαίνεται με supervised; Τρεις τύποι insight που το Q2 (supervised) δεν παράγει:

- **Ετερογένεια του BENIGN.** Το DBSCAN σπάει το BENIGN σε πολλά καθαρά pockets - η «κανονική» κίνηση είναι μείγμα πολλών προφίλ. Στο supervised setting ο classifier «μαθαίνει» το BENIGN ως μία class αλλά πρακτικά πρέπει να μάθει πολλά distinct flavors.
- **Outliers / unknown attacks.** Τα noise points του DBSCAN είναι υποψήφια για zero-day attacks ή corrupted flows που δεν ταιριάζουν σε καμία πυκνή ομάδα.
- **Η κυρίαρχη διχοτόμηση των δεδομένων.** Ο silhouette-best $k = 2$ (normal vs DoS/DDoS) μας λέει ότι το πιο ισχυρό σήμα στα 47 features είναι η διάκριση ογκώδους επίθεσης από κανονική κίνηση - πληροφορία που στο supervised setting «κρύβεται» πίσω από τις 15 ετικέτες.

5.5.2 Σύγκριση Proportional vs Balanced Sampling (mirror του Q2)

Ως καθρέφτη του Q2, ξανατρέξαμε ολόκληρο το Q3 pipeline με **BALANCED_SAMPLING=True** στο ίδιο **TARGET_ROWS=500_000**. Εξισορροπώντας την πυκνότητα ανά κλάση, οι rare attacks αποκτούν αρκετά σημεία ώστε να αναδειχθούν γεωμετρικά - και πράγματι το silhouette-best k μετατοπίζεται από 2 (proportional) σε 12 (balanced), δηλαδή κοντά στον αριθμό των πραγματικών labels (15). Κρίσιμη διαφορά είναι η ίδια η μορφή της καμπύλης silhouette: στο proportional δείγμα έχει global maximum στο $k = 2$ και μετά πέφτει· στο balanced δείγμα ανεβαίνει από $k = 2$ (0,391) και κορυφώνεται στο $k = 12$ (0,481).

Algorithm	Config	Clusters	Noise	Sil.	DB
K-Means	$k = 12$ (best by silhouette)	12	0	0,4611	0,7911
K-Means	$k = 15$ (reference)	15	0	0,4710	0,7091
Hierarchical	average, $k = 12$	12	0	0,7004	0,2625
Hierarchical	complete, $k = 12$	12	0	0,6294	0,5151
Hierarchical	ward, $k = 12$	12	0	0,5126	0,7815
DBSCAN	$\varepsilon = 0,3, ms = 20$	407	27.583	0,2999	0,6564

Πίνακας 5.5: Q3 balanced run (BALANCED_SAMPLING=True, 500K) - σύγκριση best configurations ανά αλγόριθμο. Best- k μετατοπίζεται από 2 (proportional) σε 12. Το Hierarchical εκπαιδεύτηκε σε 2.468-row subsample.

Τι αλλάζει.

- **K-Means:** best $k = 12$ (αντί 2) με silhouette 0,461· το $k = 15$ reference δίνει 0,471 - πλέον οι δύο τιμές είναι σχεδόν ίσες (στο proportional το $k = 15$ υστερούσε σαφώς έναντι του $k = 2$). Η εξισορρόπηση κάνει τη δομή « $\approx$ #labels» ανταγωνιστική.
- **DBSCAN:** το best silhouette σχεδόν τριπλασιάζεται (0,300 στο $\varepsilon = 0,3, ms = 20$ έναντι 0,105 στο proportional), αλλά με πολύ περισσότερο noise (27.583 σημεία, $\sim 5,5\%$, έναντι 2.649): χωρίς την κυριαρχία του BENIGN τα σύνορα πυκνότητας γίνονται καθαρότερα αλλά απομονώνονται περισσότερα border points.
- **Hierarchical:** το best linkage γίνεται *average* (sil 0,700, DB 0,263) σε $k = 12$ · τα ψηλά silhouettes εξακολουθούν να αντανakλούν εν μέρει το μικρό (2.468-row) subsample, οπότε διαβάζονται με την ίδια επιφύλαξη όπως στο main run.
- **PCA:** ουσιαστικά αμετάβλητο (19 components για $\geq 85\%$, cum 0,879) - η γραμμική δομή των features δεν εξαρτάται από τη στρατηγική δειγματοληψίας.

Συμπέρασμα (το mirror). Στο unsupervised Q3 το proportional είναι το ειλικρινές (το clustering πρέπει να δει την πραγματική πυκνότητα). Το balanced run εδώ δεν είναι το «σωστό» αποτέλεσμα - είναι το εργαλείο που αποκαλύπτει ότι η αντιστοίχιση cluster $\leftrightarrow$ attack υπάρχει αλλά κρύβεται κάτω από τα 83% BENIGN του ρεαλιστικού δείγματος.

5.6 Σύνοψη Q3

Μέγεθος	Τιμή
Sample (proportional, TARGET_ROWS=500K)	500.000 rows
PCA components ($\geq 85\%$ variance)	19 (cum 0,863)
Hierarchical subsample	2.468 rows
K-Means k-search	$k \in [2, 15]$
K-Means best k (silhouette)	2 (reference: 15)
K-Means metrics (k=2)	sil 0,5653 / DB 1,4935
Hierarchical best linkage (k=2)	complete/average
Hierarchical metrics	sil 0,9201 / DB 0,2488
DBSCAN sweep	$5 \times 3 = 15$ combos
DBSCAN best (silhouette)	$\varepsilon = 1,5, ms = 5$
DBSCAN best metrics	sil 0,1051 / DB 0,5493
DBSCAN noise points (best)	2.649 ($\sim 0,5\%$)

Πίνακας 5.6: Βασικά αποτελέσματα του Q3 (proportional 500K run).

Κεφάλαιο 6

Συμπεράσματα

6.1 Σύνοψη Όλου του Αγωγού

Το παρόν project υλοποίησε end-to-end ML αγωγό πάνω στο CIC-IDS-2017: EDA + cleaning (Q1), supervised classification (Q2) και unsupervised clustering (Q3). Όλη η αναπαραγωγή στηρίζεται σε ένα κοινό cached Parquet snapshot (`data/cic_ids_2017_clean.parquet`, 221,9 MB) και μία ενιαία `RANDOM_STATE = 42`.

6.2 Συμπεράσματα ανά Ερώτημα

6.2.1 Q1 - Δομή και Καθαρισμός των Δεδομένων

1. Το **CIC-IDS-2017** απαιτεί προσεκτικό καθαρισμό πριν οποιοδήποτε μοντέλο: 4.376 Infs, 255.236 exact duplicates και corrupted bytes στις ετικέτες των Web Attacks.
2. Η **ανισορροπία είναι ακραία** ($\sim 195\,000 : 1$). Αυτό διαμόρφωσε κάθε επιλογή στο Q2 (class_weight, macro-F1, stratified splitting) και στο Q3 (per-class subsampling cap).
3. Το **feature space είναι υπερ-παραμετροποιημένο**. Οι 78 αρχικές αριθμητικές στήλες έκρυναν 22 near-duplicates και 8 constants. Μετά την αφαίρεση μένουν **47 features** με γνήσια διακριτική πληροφορία.
4. Η χρήση **Parquet** για το cached snapshot μειώνει το I/O cost από ~ 60 sec (raw CSV parsing) σε < 2 sec.

6.2.2 Q2 - Supervised Classification

1. Το **Decision Tree είναι ο νικητής** (F1-weighted 0,9973, F1-macro 0,8705) - υψηλότερο και στις δύο μετρικές από RF (0,9927 / 0,7403) και LR (0,9548 / 0,6703). Ένα βαθύ δέντρο κόβει καθαρά τις rare κλάσεις· το LR υστερεί στο macro επειδή οι attack classes δεν είναι γραμμικά διαχωρίσιμες στο 47-feature space.
2. **Balanced sampling είναι κρίσιμο**. Με proportional (83% BENIGN) το LR macro κατέρρευσε σε 0,48· με balanced ανέβηκε σε 0,67 (+0,19), το RF +0,07. Επιβεβαίωση του κανόνα: το training set πρέπει να αναπαριστά τα *σύνορα* των κλάσεων.
3. Το **split-vs-CV protocol δίνει ουσιαστικά ίδια best params** (μεγάλο val set 100K). Η CV είναι πιο αξιόπιστη ως ranking αλλά πολύ πιο αργή (RF-B 2174s vs RF-A 362s).
4. Η **Destination Port dominate ένα μεμονωμένο DT** (importance $\sim 0,46$). Στο RF η importance εξομαλύνεται μέσω bagging - πέφτει σε $\sim 0,055$, top-10 στη ζώνη $\sim 0,04$ -0,05.
5. Configurable mode (TARGET_ROWS + BALANCED_SAMPLING): 150_000 (fast) / 500_000 (το run που παρουσιάζεται) / None (full). Το LR με `lbfgs` είναι ο κύριος bottleneck· επιταχύνσεις CPU/GPU στο Κεφάλαιο [B'](#).

6.2.3 Q3 - Unsupervised Clustering

1. **19 PCA components** καλύπτουν $\sim 86,3\%$ της variance. Δεν υπάρχει ένα κυρίαρχο axis - η information είναι κατανεμημένη.
2. **To silhouette-best k για το K-Means είναι 2**, όχι 15: η ισχυρότερη γεωμετρική δομή υπό proportional (83% BENIGN) sampling είναι μια δυαδική διχοτόμηση *normal vs volumetric DoS*. Αυτό **δεν** σημαίνει ότι υπάρχουν μόνο 2 attack types - σημαίνει ότι οι 15 κλάσεις δεν είναι καθαρά διαχωρίσιμες ως 15 γεωμετρικές «μπάλες»· η λεπτότερη δομή υπάρχει αλλά είναι επικαλυπτόμενη (ένα ειλικρινές εύρημα).
3. **To Hierarchical έδωσε το υψηλότερο silhouette** (complete/average 0,920 σε 2.468-row subsample λόγω $O(n^2)$ memory) - αλλά *παραπλανητικά*: σε $k = 2$ αντανακλά έναν εύκολο διαχωρισμό, όχι αντιστοίχιση με labels. Το silhouette πρέπει πάντα να διασταυρώνεται με το περιεχόμενο των clusters.
4. **DBSCAN δυσκολεύεται** σε αυτό το πυκνό dataset: best silhouette μόλις 0,105 ($\epsilon = 1,5$, $ms = 5$), με αρνητικά scores στα μεγάλα ϵ . Τα noise points παραμένουν candidate *outliers / unknown attacks* - πληροφορία που το supervised setting του Q2 δεν παράγει.
5. **Η BENIGN class διασπείρεται σε πολλά clusters**, ενώ χαρακτηριστικές attack υπογραφές (DDoS, PortScan, FTP-Patator) σχηματίζουν compact και σχεδόν καθαρά clusters. Δηλαδή το κανονικό traffic είναι μείγμα από web/email/SSH/bulk flavors, ενώ οι attacks έχουν διακριτές statistical signatures.

6.3 Συνολικά Διδάγματα και Best Practices

- **Pipeline-driven preprocessing.** Σε όλο το Q2 και Q3, κάθε scaling και dimensionality reduction βρίσκεται μέσα σε Pipeline ή εφαρμόζεται μόνο στο training fold ώστε να αποφευχθεί data leakage.
- **Reproducibility.** `utils.RANDOM_STATE = 42` χρησιμοποιείται σε κάθε split, model seed, και stochastic operation (sampling, K-Means init, RF bootstrap, GridSearchCV folds, PCA solver). Re-execution οποιουδήποτε notebook top-to-bottom δίνει bit-identical αποτελέσματα.
- **Caching.** Το Q1 παράγει το 221,9 MB Parquet snapshot που και τα Q2 και Q3 φορτώνουν σε < 2 sec - όχι re-execution του πλήρους Q1 αγωγού σε κάθε run.
- **Configurable compute mode.** Q2 και Q3 εκθέτουν `TARGET_ROWS` και `BALANCED_SAMPLING` ώστε ο χρήστης να επιλέγει μέγεθος και στρατηγική δειγματοληψίας χωρίς αλλαγές αλλού.
- **Hardware acceleration με fallback.** Το Q2 χρησιμοποιεί Intel `scikit-learn-intelex` (CPU) και το Q3 `RAPIDS cuML` (GPU), πάντα μέσα σε `try/except` ώστε τα notebooks να τρέχουν και χωρίς αυτά (Κεφάλαιο B').
- **Διαχωρισμός responsibilities.** Reusable helpers ζουν στο `utils.helpers` (re-exported μέσω `utils/__init__.py`). Notebooks είναι αμιγώς διδακτικά - Greek Γιατί cells με τεκμηρίωση κάθε απόφασης.

6.4 Πιθανές Επεκτάσεις

- **Σύγχρονοι classifiers:** XGBoost, LightGBM ή gradient boosting θα μπορούσαν να βελτιώσουν περαιτέρω τα macro-F1 metrics (RF μας δίνει 0,912).
- **Threshold tuning:** για production IDS, η εξάρτηση από F1-macro vs precision/recall ζευγάρια

ανά class θα οδηγούσε σε διαφορετική τοποθέτηση thresholds.

- **Σύγχρονα clustering:** HDBSCAN (αυτόματη επιλογή ϵ , χειρίζεται variable density) ή UMAP-based clustering θα μπορούσαν να βελτιώσουν τη DBSCAN υλοποίηση - ιδίως δεδομένου ότι το DBSCAN δυσκόλεψε στο πυκνό μας dataset (silhouette 0,105).
- **Mini-batch / streaming K-Means** για εκπαίδευση σε ολόκληρο το dataset (2,5M) χωρίς subsampling· δοκιμάστηκε `SGDClassifier` για το LR αλλά αποκλίνει από το `lbfgs` (βλ. [B'](#)).
- **Online / streaming setup:** το CIC-IDS-2017 είναι static· σε production setting, η ανίχνευση πρέπει να γίνεται σε real-time πάνω σε streaming flows.

Παράρτημα Α΄

Οδηγίες Εγκατάστασης και Εκτέλεσης

Α΄.1 Απαιτήσεις

- Python 3.10+ (δοκιμάστηκε σε 3.12).
- ~ 16 GB RAM (το full load απαιτεί ~ 4-6 GB μετά το `_optimize_memory`).
- ~ 3 GB ελεύθερου χώρου (raw CSV + Parquet cache).
- Προαιρετικά για επιτάχυνση: Intel CPU (για το `scikit-learn-intelex`, Q2) και/ή NVIDIA GPU με CUDA 12+ μέσω WSL2/Linux (για το `RAPIDS cuML`, Q3). Και τα δύο είναι προαιρετικά - τα notebooks κάνουν fallback σε stock scikit-learn. Πλήρεις οδηγίες setup στο Κεφάλαιο Β΄.

Α΄.2 Εγκατάσταση

```
Εγκατάσταση
1 # clone και venv
2 git clone https://github.com/IBilba/network-intrusion-detection-ml.git
3 cd network-intrusion-detection-ml
4 python -m venv .venv
5 source .venv/bin/activate # ή .venv\Scripts\activate στα Windows
6
7 # dependencies
8 pip install -r requirements.txt
9
10 # data τοποθέτηση( των 8 CSV στο data/)
11 # από Kaggle: https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset/
```

Α΄.3 Εκτέλεση του Q1

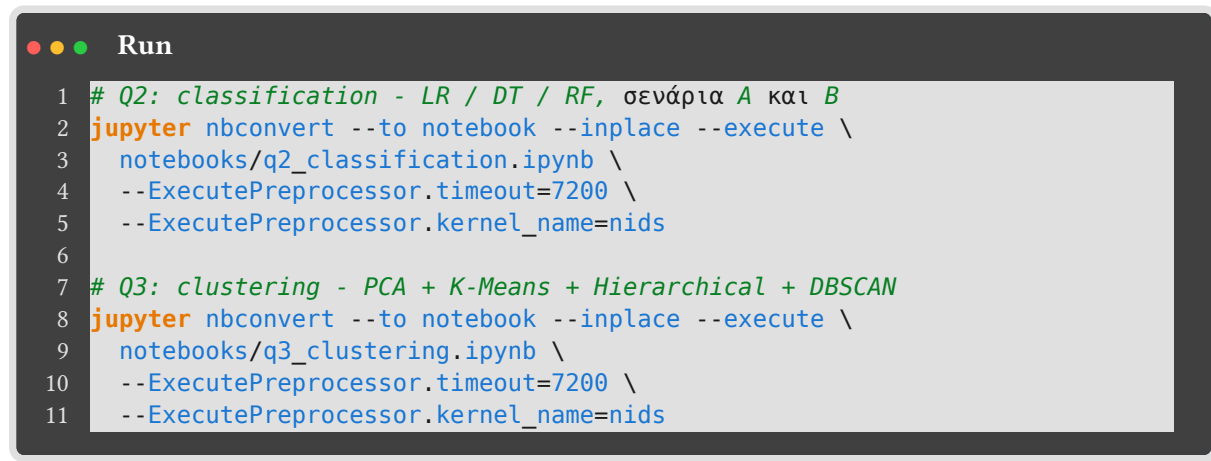
```
Run
1 # Εκτέλεση end-to-end γράφει( figures, CSV, Parquet)
2 jupyter nbconvert --to notebook --execute notebooks/q1_eda.ipynb \
3 --output q1_eda.ipynb
```

Μετά την εκτέλεση, τα παραγόμενα αρχεία βρίσκονται σε:

- `outputs/figures/`: 5 PNGs (label distribution, histograms, boxplots, heatmap).
- `outputs/results/`: 7 CSVs (column catalogue, describe, label distribution, correlation με label, top correlated pairs, removed features, final feature list).
- `data/cic_ids_2017_clean.parquet`: το cached clean dataset για τα Q2/Q3.

Α΄.4 Εκτέλεση των Q2 και Q3

Τα Q2 και Q3 διαβάζουν μόνο το cached Parquet snapshot - δεν ξανατρέχουν το pipeline του Q1. Πριν την εκτέλεση, ανοίξτε το αντίστοιχο notebook και προσαρμόστε τις μεταβλητές **TARGET_ROWS** (150_000 / 500_000 / None) και **BALANCED_SAMPLING** (Q2: True, Q3: False ως default). Για το GPU-accelerated Q3, εκτελέστε το στο WSL2 `rapids-gpu kernel` (Κεφάλαιο Β΄).



```
Run
1 # Q2: classification - LR / DT / RF, σενάρια A και B
2 jupyter nbconvert --to notebook --inplace --execute \
3   notebooks/q2_classification.ipynb \
4   --ExecutePreprocessor.timeout=7200 \
5   --ExecutePreprocessor.kernel_name=nids
6
7 # Q3: clustering - PCA + K-Means + Hierarchical + DBSCAN
8 jupyter nbconvert --to notebook --inplace --execute \
9   notebooks/q3_clustering.ipynb \
10  --ExecutePreprocessor.timeout=7200 \
11  --ExecutePreprocessor.kernel_name=nids
```

Παραγόμενα αρχεία:

- **Q2:** 13 PNGs (6 confusion matrices, 2 feature-importance bars, 1 F1 comparison, 1 LR coefficient heatmap, 2 DT visualizations, 1 RF single-tree) + 12 CSVs (subsample, split, label encoding, 6 grid CSVs, 2 feature-importance, comparison table, label encoding).
- **Q3:** 11 PNGs (PCA variance, K-Means elbow/silhouette, 2 K-Means scatters, K-Means heatmap, dendrogram, hier scatter + heatmap, k-distance, DBSCAN scatter + heatmap) + 8 CSVs (PCA variance, k-search, linkage comparison, DBSCAN sweep, comparison, 3 heatmap composition tables).

Α΄.5 Αναπαραγωγιμότητα

Όλες οι στοχαστικές λειτουργίες (sampling, shuffling, CV folds) θα χρησιμοποιούν τον ίδιο **RANDOM_STATE = 42** από το `utils/helpers.py`. Με τον ίδιο seed και το ίδιο σύνολο δεδομένων, όλοι οι αριθμοί του Πίνακα 3.6 είναι bit-exactly αναπαραγώγιμοι.

Παράρτημα Β΄

Επιτάχυνση Υλικού και Βελτιστοποίηση

Για να γίνει εφικτή η εκπαίδευση σε μεγαλύτερα δείγματα (έως 500K–1M rows), διερευνήσαμε CPU και GPU επιταχύνσεις. Αυτό το παράρτημα καταγράφει τι χρησιμοποιήσαμε, γιατί, και πώς αναπαράγεται – μαζί με τα σημαντικά αρνητικά ευρήματα (πού μια επιτάχυνση *άλλαξε* τα αποτελέσματα και απορρίφθηκε).

Β΄.1 Hardware και Επισκόπηση

Δοκιμές σε: Intel Core i7-10700 (8c/16t), NVIDIA RTX 2060 (6 GB VRAM), 16 GB RAM, Windows + WSL2 (Ubuntu 24.04). Τελική κατανομή:

- **Q2 (classification):** stock scikit-learn 1.8 + `scikit-learn-intelx` (Intel oneDAL, CPU).
- **Q3 (clustering):** RAPIDS cuML 26.04 (GPU) για PCA / K-Means / DBSCAN, με fallback σε stock scikit-learn· το Hierarchical μένει σε CPU (scipy/sklearn).

Β΄.2 Το Βασικό Δίδαγμα: οι Accelerators δεν είναι πάντα Ισοδύναμοι

Ένα «drop-in» accelerator μπορεί να αλλάξει τα *αποτελέσματα* ενός αλγορίθμου, όχι μόνο την ταχύτητα. Το επαληθεύσαμε τρεις φορές, πάντα με A/B test έναντι stock scikit-learn *πριν* την υιοθέτηση:

- **intelx K-Means (Q3) – ΑΠΟΡΡΙΦΘΗΚΕ.** Παρήγαγε degenerate split (ένα cluster ~ 24 ακραίων σημείων με τεχνητό silhouette $\sim 0,95$) που χάλαγε την επιλογή best- k . Γι' αυτό το Q3 K-Means/PCA έτρεξε σε cuML ή stock, ποτέ intelx.
- **cuML K-Means default init – ΔΙΟΡΘΩΘΗΚΕ.** Το default `scalable-k-means++` έδινε το ίδιο degenerate split· με `init='k-means++'` ταυτίζεται με το stock sklearn. Προσθήσαμε και runtime auto-guard (`kmeans_result_is_pathological`) που κάνει refit σε CPU μόνο όταν ένα αποτέλεσμα είναι πραγματικά degenerate (μικροσκοπικό cluster και silhouette $> 0,9$).
- **cuML / SGD Logistic Regression (Q2) – ΑΠΟΡΡΙΦΘΗΚΑΝ.** Και τα δύο είναι ταχύτερα αλλά αποκλίνουν από το sklearn `lbfgs` στα βαθμολογούμενα metrics (διαφορά f1-macro $\sim 0,15$, λόγω διαφορετικού solver/regularization). Για βαθμολογούμενο classification η αναπαραγωγιμότητα υπερσχύει της ταχύτητας, οπότε το Q2 έμεινε σε CPU+intelx (το intelx είναι bit-ταυτόσημο με sklearn).

Β'.3 Μετρήσεις Επιτάχυνσης

Εργασία	Backend	Χρόνος
Q3 DBSCAN sweep (15 combos, 150K)	stock sklearn (CPU)	~ 58 min
Q3 DBSCAN sweep (15 combos, 150K)	cuML (GPU)	~ 2 min
Q3 DBSCAN sweep (15 combos, 500K)	cuML (GPU)	~ 23 min
Q3 K-Means k-search + PCA (1M)	cuML (GPU)	~ 2 s + 0,2 s

Πίνακας Β'.1: Ενδεικτικές μετρήσεις. Το DBSCAN στη GPU είναι $\sim 100\times$ ταχύτερο στο χειρότερο config· τα PCA/K-Means σχεδόν στιγμιαία ακόμη και σε 1M rows (peak VRAM $\sim 1,4$ GB).

Β'.4 Όρια Κλιμάκωσης (Scaling Walls)

Δεν κλιμακώνονται όλα τα βήματα το ίδιο:

- **DBSCAN είναι** $O(n^2)$ (το cuML χρησιμοποιεί brute-force neighbor graph). Από 150K $\rightarrow$ 500K rows ο χρόνος $\times 12$ (τετραγωνικά). Η GPU δίνει τεράστιο σταθερό factor αλλά δεν αλλάζει την κλάση πολυπλοκότητας – το DBSCAN δεν συμφέρει $> 500K$.
- **Hierarchical είναι** $O(n^2)$ **σε μνήμη**: 3K rows $\rightarrow$ 0,07 GB, 50K $\rightarrow$ 20 GB, 100K $\rightarrow$ 80 GB. Υποχρεωτικά τρέχει σε μικρό subsample (~ 2.468 rows) – δεν υπάρχει GPU λύση.
- **Silhouette είναι** $O(n^2)$: υπολογίζεται πάντα σε sample (20K rows), σε κάθε hardware. **Davies-Bouldin είναι** $O(n)$: τρέχει σε όλα τα δεδομένα.

Β'.5 Setup του RAPIDS cuML (WSL2) – Αναπαραγωγή

Το cuML δεν έχει build για Windows· απαιτεί Linux ή WSL2.

```
WSL2 + RAPIDS cuML
1 # μέσα( σε WSL2 Ubuntu, με NVIDIA driver >= 525 / CUDA 12)
2 sudo apt install -y python3.12-venv python3-pip
3 python3 -m venv ~/rapids-env && source ~/rapids-env/bin/activate
4 pip install --upgrade pip
5 # cuML (CUDA-12 wheels) + notebook stack
6 pip install --extra-index-url=https://pypi.nvidia.com cuml-cu12
7 pip install jupyter ipykernel pandas pyarrow scikit-learn matplotlib
  seaborn scipy
8 python -m ipykernel install --user --name rapids-gpu \
9   --display-name "Python (RAPIDS GPU)"
10 # Χρήση από VS Code (Windows): jupyter lab --no-browser --port=8888
11 # -> "Existing Jupyter Server" -> paste localhost URL -> kernel rapids-
  gpu
```

Επιβεβαίωση: το πρώτο cell του Q3 τυπώνει DBSCAN backend: cuML (GPU) και PCA/KMeans backend: cuML (GPU). Σε μηχανήμα χωρίς cuML, τυπώνει scikit-learn (CPU) και το notebook τρέχει κανονικά.

B'6 Στρατηγική Δειγματοληψίας

Η συνάρτηση `subsample_to_target(y, target_rows, target_frac, rare_floor, balanced)` ελέγχει το μέγεθος και τη στρατηγική:

- **balanced=True** (Q2): ίσο ceiling στις μεγάλες κλάσεις – αναγκαίο για να μάθει ο classifier τα σύνορα όλων των κλάσεων.
- **balanced=False** (Q3): proportional – διατηρεί την πραγματική πυκνότητα ώστε το clustering να βρει τη φυσική δομή.

Αυτή η αντίθετη επιλογή ανά ερώτημα είναι σκόπιμη: supervised και unsupervised tasks έχουν αντίθετες ανάγκες δειγματοληψίας. Η ποσοτική επίδραση φαίνεται στον Πίνακα [4.5](#).